FACULTY OF COMPUTING AND INFORMATION TECHNOLOGY

BMDS2003 Data Science

Assignment Title:

Apartment Rent

Semester 202605

Programme (Year & Semester)	**:**	RDSY2S1
Tutorial Group	**:**	RDSG4
Tutor's Name	**:**	YEN WEI YAN

Team members:

No	Name	Registration No.	Signature
1	Lee Zhe Hui	26WMR12501	zeo
2	Neo Chen Heng	26WMR12515	heng
3	Tan Ying Nan	26WMR12539	Tan
4	Wong Ying Yi	26WMR12555	yingyi

Table of Contents

1.0 Executive Summary

The residential rental market is complex because rental prices can vary significantly depending on location, property characteristics, and available amenities. Traditional rental price estimation methods often rely on local market knowledge or simple linear assumptions, which may not fully capture the complex relationships between geographic factors, property size, and premium features.

This project applies the Cross-Industry Standard Process for Data Mining (CRISP-DM) framework (Chapman et al., 2000) to develop a machine learning-based rental price prediction system using a nationwide rental classified ads dataset containing 99,492 listings from 50 U.S. states.

A complete data preparation and modelling pipeline was developed, including data cleaning, missing value handling, outlier treatment, feature engineering, target transformation, and feature scaling. These preprocessing steps were performed to improve data quality and ensure that the models could learn meaningful patterns from the dataset. Four machine learning regression models were trained and evaluated:

1. Linear Regression (Baseline Model)
2. Decision Tree Regressor (Tuned: Depth = 16, Min Leaf = 10)
3. Random Forest Regressor (100 Trees Bagging Ensemble, Max Depth = 25)
4. Histogram-Based Gradient Boosting Regressor (Tuned: Learning Rate = 0.08, Max Leaf Nodes = 127, Early Stopping)

The evaluation results showed that tree-based models achieved better performance compared with the Linear Regression baseline because they can capture non-linear relationships within the rental data. Linear Regression achieved an R^2 score of 0.6580 and an MAE of $217.92. The Decision Tree improved the performance with an R^2 score of 0.7381 and an MAE of $185.87. Random Forest further reduced prediction errors by using multiple decision trees, achieving an R^2 score of 0.8309 and an MAE of $140.62. The Histogram-Based Gradient Boosting Regressor achieved the best overall performance with an R^2 score of 0.8451, MAE of $141.21, and MAPE of 10.52%. In addition, 86.7% of its predictions were within ±20% of the actual rental prices.

The final selected model was deployed through an interactive Streamlit web application that allows users to estimate rental prices based on property information, including location, size, layout, and amenities. The project also used MLflow for experiment tracking and stored the results in a SQLite database to support model management and future reference.

2.0 Business Understanding

2.1 Industry Background and Organizational Context

Real estate property managers, investors, tenants, and property owners often face difficulties when determining suitable rental prices. Unlike residential sales markets that usually have more standardized transaction records, rental listings are often fragmented and highly dependent on local market conditions.

Property managers may set rental prices too high, resulting in longer vacancy periods, or too low, causing potential loss of rental income. Therefore, a data-driven rental price prediction system can provide more objective pricing guidance by considering important factors such as property size, location, layout, and available amenities.

The target users of this rental valuation system include:

- Property Managers and Landlords: To estimate competitive rental prices based on market data.
- Real Estate Investors: To evaluate potential rental returns before making investment decisions.
- Prospective Tenants: To compare rental prices and identify whether a listing is reasonably priced.
- Real Estate Platforms: To provide automated pricing recommendations for rental listings.

2.2 Problem Statement and Analytical Objectives

The primary analytical problem addressed in this project is the accurate estimation of monthly apartment rental rates based on multi-dimensional property attributes. Rental prices are affected by various factors, including physical property characteristics such as living area, number of bedrooms, and bathrooms, as well as geographic factors, amenities, and listing-related information.

Due to the interaction between these factors, traditional linear methods may not be able to fully capture the complex and non-linear relationships that influence rental prices. Therefore, machine learning models are applied in this project to identify hidden patterns within the rental dataset and improve the accuracy of rental price prediction.

The analytical objectives of this project are as follows:

1. Exploratory Pattern Discovery - To perform exploratory data analysis (EDA) to identify important relationships, rental price patterns, geographic variations, and unusual observations within the dataset. The analysis helps to understand the factors that may influence apartment rental prices before developing the machine learning models.
2. Data Preparation - To develop an appropriate preprocessing pipeline by handling missing values, treating outliers, transforming variables, and preparing the dataset for machine learning modelling. These steps ensure that the data is in a suitable format and can improve the performance of the prediction models.
3. Model Development and Comparison - To develop and compare four regression models, starting from a baseline model and progressing to more advanced tree-based ensemble models. The comparison is conducted to determine which model provides better prediction performance for estimating apartment rental prices.
4. Model Performance Evaluation - To evaluate and compare the performance of the developed models using regression evaluation metrics, including R^2, MAE, RMSE, MAPE, and prediction tolerance ranges. These evaluation results are used to identify the best-performing model.
5. Deployment Prototype Development - To develop a Streamlit application as a deployment prototype that allows users to estimate apartment rental prices based on property information such as location, size, layout, and amenities.

2.3 Requirements, Assumptions, and Constraints

Project Requirements:

- Develop and compare four machine learning regression models for apartment rental price prediction.
- Include a baseline regression model to provide a performance benchmark for evaluating advanced models.
- Select the best-performing model based on appropriate regression evaluation metrics.
- Develop a Streamlit-based deployment prototype for practical rental price prediction.

Assumptions:

- The dataset provides a reasonable representation of the US rental market during the data collection period.
- Rental prices recorded in the dataset are assumed to represent actual monthly rental values.
- Location information, including city and geographic coordinates, is assumed to be accurate.

Constraints:

- The model development process is limited by consumer-grade computational resources (Apple MacBook Air M-series).
- The dataset represents a static snapshot and does not capture future changes in rental market conditions.
- External factors such as economic changes, inflation, and housing policies are not included in the dataset.

2.4 Data Mining Success Criteria

Table 2.1: Data Mining Success Criteria

Criterion	Target Threshold	Rationale
Coefficient of Determination ((R²)	> 0.80	Model should explain at least 80% of the variation in rental prices
Mean Absolute Error (MAE)	< $150.00 USD	Average prediction error should remain below $150 USD.
Mean Absolute Percentage Error (MAPE)	< 12.0%	Maintain prediction errors within an acceptable range for rental price estimation
Prediction Accuracy within ±20%	> 85.0%	Over 85% of predictions within ± 20% of actual rent
Deployment Prototype	Functional Streamlit App	Real-time sub-second inference

2.5 Resource Inventory and Environment Setup

The project was developed using Python 3.13 within an isolated virtual environment (.venv).

Software Stack:

- Data Processing: pandas (McKinney, 2010), numpy, scipy
  Visualization: matplotlib, seaborn
- Machine Learning: scikit-learn (sklearn) (Pedregosa et al., 2011)
- Experiment Tracking: mlflow (Zaharia et al., 2018)
- Model Serialization: joblib
- Web Deployment: streamlit

Hardware & OS: Apple MacBook Air (M-series ARM64), macOS / Windows (cross-platform compatible development environment)

2.6 Technical Risks and Mitigation Strategies

Several technical challenges were considered during the development process. The following risks and mitigation strategies were identified to improve model reliability and reduce potential errors.

Table 2.2: Technical Risks and Mitigation

Risk	Severity	Mitigation
Data Leakage	High	Perform strict train-test partitioning before applying target encoding and feature standardization
Target Skewness	High	Apply logarithmic transformation: y_log = ln(1 + price) to reduce the impact of highly skewed rental prices
High Cardinality (2900+ cities)	Medium	Implement out-of-fold city median target encoding
Multicollinearity	Medium	Use non-linear tree-based models inherently robust to collinearity
Imbalanced Geographic Distribution	Low	Target encoding with global median fallback for underrepresented cities

3.0 Data Understanding

3.1 Dataset Source and Metadata Overview

The underlying data for this study was obtained from the UCI Machine Learning Repository ("Apartment for Rent Classified" dataset, published by RentDigs.com). This dataset provides a detailed representation of the residential rental market, allowing comprehensive analysis of pricing patterns, geographic variations, and property characteristics.

Dataset Specifications:

- Dataset Source: UCI Machine Learning Repository / RentDigs.com (100K classified rental records)
- Total Records: 99,492 observations
- Total Columns: 22 features
- File Format: CSV (semicolon-delimited, CP1252 encoding)
- File Size: ~102 MB
- Geographic Coverage: 50 US states + District of Columbia
- City Coverage: 2,900+ unique city names
- Target Variable: price (Monthly apartment rent in USD)

3.2 Feature Dictionary and Attribute Types

Table 3.1: Complete Feature Dictionary of Raw Dataset Attributes

#	Feature Name	Raw Data Type	Variable Type	Description
1	id	int64	Identifier	Unique listing identifier (to be dropped)
2	category	object	Categorical	Housing category
3	title	object	Text	Title of the listing advertisement
4	body	object	Text	Full descriptive text body
5	amenities	object	Text	Comma-delimited list of apartment amenities
6	bathrooms	float64	Numerical (Discrete)	Number of bathrooms (includes half-baths)
7	bedrooms	float64	Numerical (Discrete)	Number of bedrooms (0 = Studio)

#	Feature Name	Raw Data Type	Variable Type	Description
8	currency	object	Categorical	Currency symbol (USD for all)
9	fee	object	Categorical	Whether broker fee is required
10	has_photo	object	Categorical	Photo availability (Yes/Thumbnail/No)
11	pets_allowed	object	Categorical	Pet policy (60.7% missing, dropped)
12	price	float64	Numerical (Continuous)	DEPENDENT (TARGET) - Monthly rental price USD
13	price_display	object	Text	Formatted price display string
14	price_type	object	Categorical	Payment frequency
15	square_feet	int64	Numerical (Continuous)	Total living space area in sq ft
16	address	object	Text	Street address (91.5% missing, dropped)
17	cityname	object	Categorical	City name (2,900+ unique values)
18	state	object	Categorical	US State abbreviation
19	latitude	float64	Numerical (Continuous)	GPS latitude coordinate
20	longitude	float64	Numerical (Continuous)	GPS longitude coordinate
21	source	object	Categorical	Listing source website
22	time	int64	Timestamp	UNIX publication timestamp

3.3 Initial Statistical Profiling

	id	bathrooms	bedrooms	price	square_feet	latitude	longitude	time
count	9.949200e+04	99429.000000	99368.000000	99491.000000	99492.000000	99467.000000	99467.000000	9.949200e+04
mean	5.358321e+09	1.445323	1.728212	1527.057281	956.430688	36.947988	-91.568656	1.559665e+09
std	1.847404e+08	0.547021	0.749200	904.245882	417.571522	4.599461	15.817168	1.105077e+07
min	5.121046e+09	1.000000	0.000000	100.000000	101.000000	19.573800	-159.369800	1.544174e+09
25%	5.197950e+09	1.000000	1.000000	1013.000000	729.000000	33.746500	-104.791900	1.550832e+09
50%	5.508673e+09	1.000000	2.000000	1350.000000	900.000000	37.228200	-84.562300	1.568745e+09
75%	5.509007e+09	2.000000	2.000000	1795.000000	1115.000000	39.953000	-77.608200	1.568767e+09
max	5.669439e+09	9.000000	9.000000	52500.000000	50000.000000	64.833200	-68.778800	1.577391e+09

Figure 3.1: Descriptive statistics summary of the raw dataset using pandas df.describe().

Key observations from the initial statistical profiling:

1. Target Variable (price):
    - Mean = $1,527.18, Median = $1,350.00, Std Dev = $904.25
    - Min = $100, Max = $52,500
    - The mean exceeding the median by $177 confirms positive right-tail skewness.
2. Living Area (square_feet):
    - Mean = 956.43 sq ft, Median = 900.0 sq ft
    - Min = 107 sq ft, Max = 40,000 sq ft
    - Extreme maximum values indicate data quality issues requiring IQR filtering.
3. Missing Value Analysis:

```
id                  0
category            0
title               0
body                0
amenities       16044
bathrooms          63
bedrooms          124
currency            0
fee                 0
has_photo           0
pets_allowed    60424
price               1
price_display       1
price_type          0
square_feet         0
address         91549
cityname          302
state             302
latitude           25
longitude          25
source              0
time                0
dtype: int64
```

Figure 3.2: Missing value counts across all 22 raw columns.

Table 3.2: Missing Value Analysis

Column	Missing Count	Missing %	Action Taken
address	~91,000	91.5%	Dropped (too sparse)
pets_allowed	~60,300	60.7%	Dropped (majority missing)
amenities	~16,000	16.1%	Rows dropped for text parsing
bathrooms	63	0.06%	Median imputation (1.0)
bedrooms	124	0.12%	Median imputation (2.0)
cityname/state	302	0.30%	Rows dropped

3.4 Exploratory Data Analysis (EDA)

3.4.1 Target Variable Analysis: Price Distribution and Log Transformation

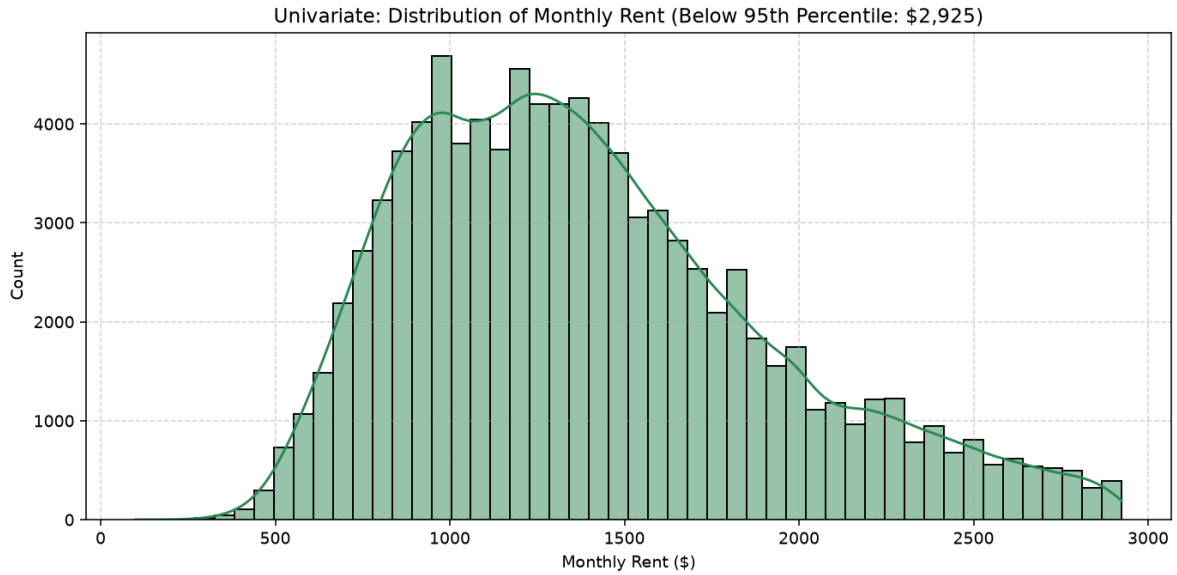


Figure 3.3: Histogram with KDE overlay showing the raw monthly rent distribution

The raw price histogram shows that most rental listings are concentrated between $800 and $2,000 per month, with the highest frequency appearing around $1,200. The distribution also shows a long right tail, which is mainly caused by higher-priced luxury apartments and possible extreme values above $4,000. This right-skewed distribution may affect regression model performance because large errors from high-priced listings can have a greater impact when using the MSE loss function.

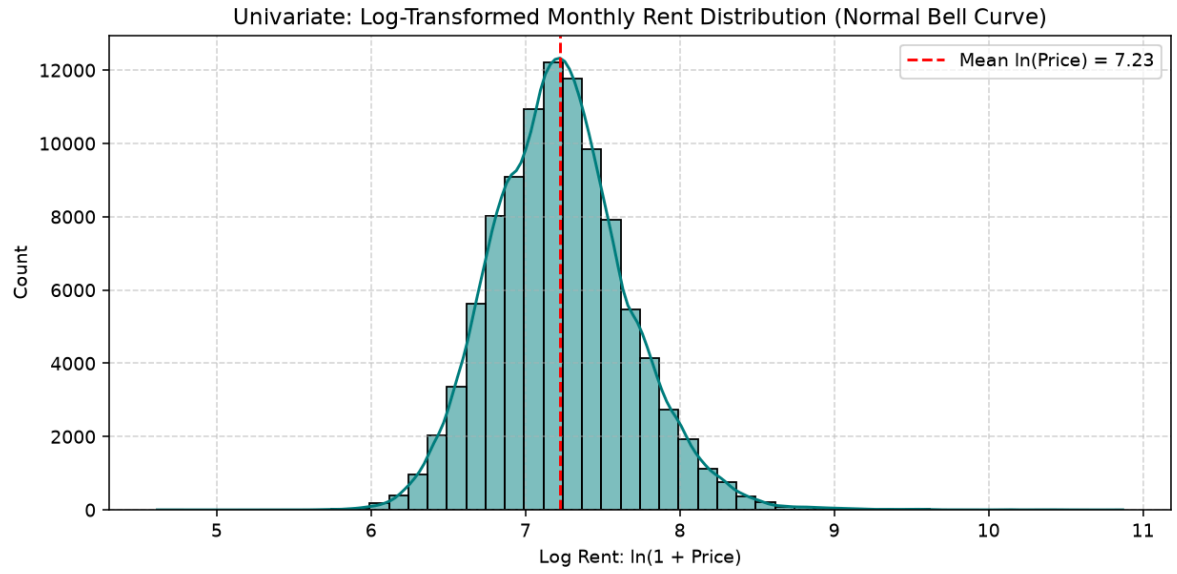


Figure 3.4: Log-transformed price distribution

After applying the natural logarithm transformation, $y_{log} = \ln(1 + \text{price})$, the distribution becomes more symmetric and closer to a normal distribution, with the centre located around 7.2 (approximately $1,340). This transformation helps reduce the effect of extreme values, improves variance stability, and makes the data more suitable for regression modelling.

3.4.2 Outlier Detection: Box Plot Analysis Before IQR Treatment

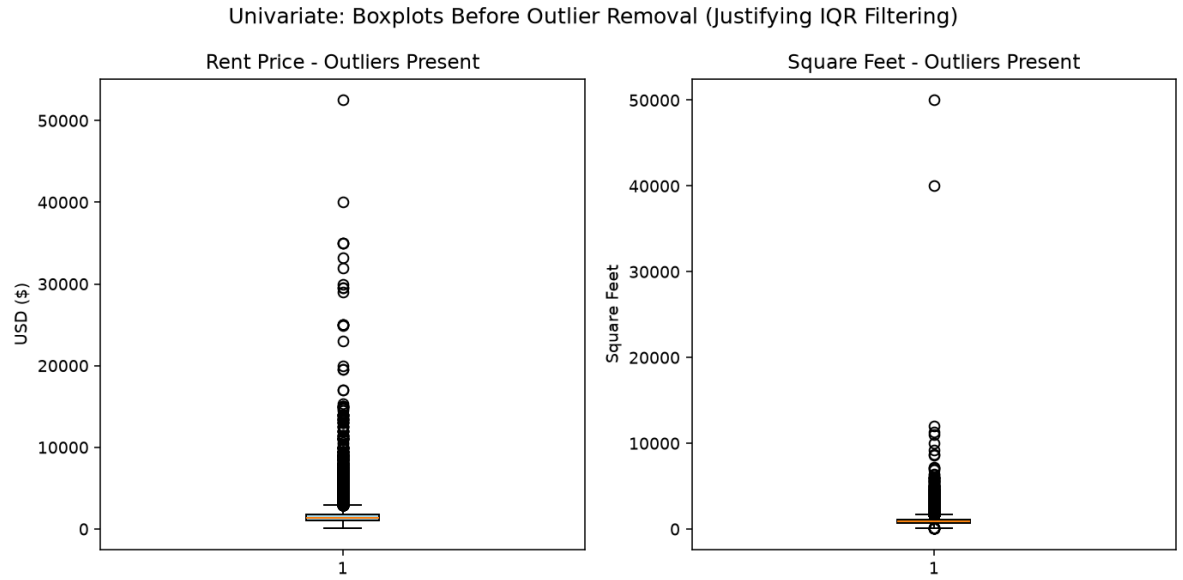


Figure 3.5: Boxplots Before Outlier Removal

The box plots show the presence of several extreme outliers in both price and square_feet. For the price variable, the IQR range is approximately between $950 and $1,850, while the upper whisker extends to around $3,200. Values above $4,500 may represent luxury properties or possible data entry errors. Similarly, square_feet contains several extremely large values above 10,000 sq ft, which are uncommon for typical residential apartments. If these outliers are not handled properly, they may affect model training by increasing the MSE loss and causing linear regression coefficients to be influenced by extreme observations.

3.4.3 Square Feet Distribution Analysis

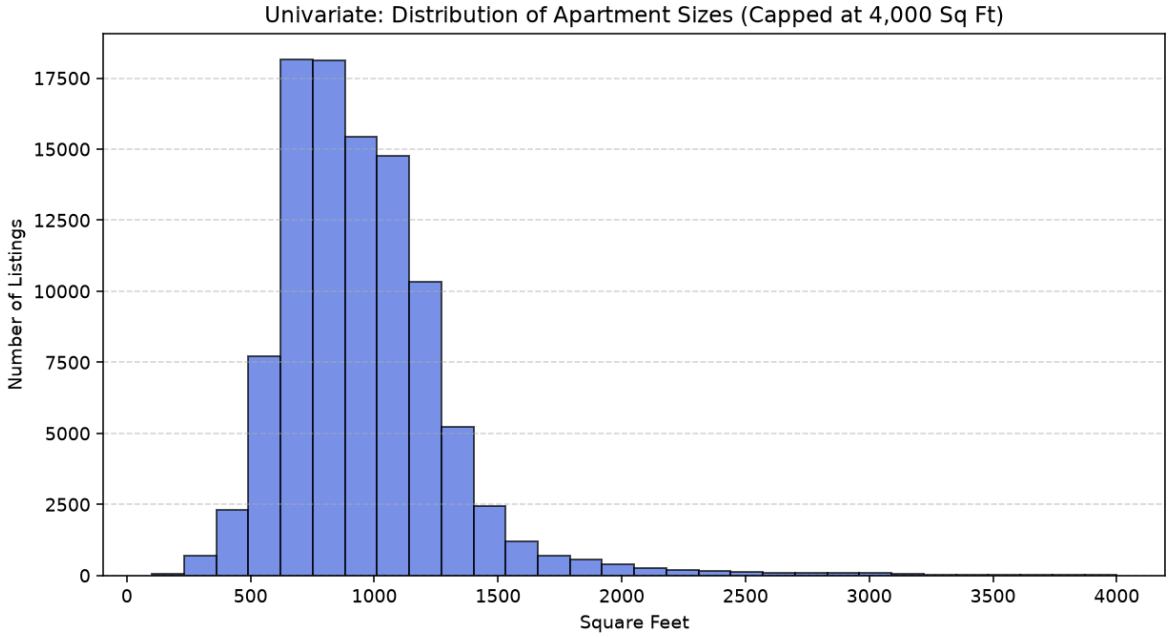


Figure 3.6: Histogram showing the distribution of apartment living area (square_feet).

The living area distribution shows that most US apartment rentals have a size between 600 and 1,400 square feet, with the highest concentration around 800–900 sq ft, which is commonly associated with 1-bedroom apartments. Only a small number of listings exceed 2,500 sq ft. Based on this observation, the square_feet variable was bounded between 150 and 4,500 sq ft during the IQR outlier treatment to reduce the influence of extreme values.

3.4.4 Categorical Feature Distribution: Bedrooms, Bathrooms, Photo, Fee

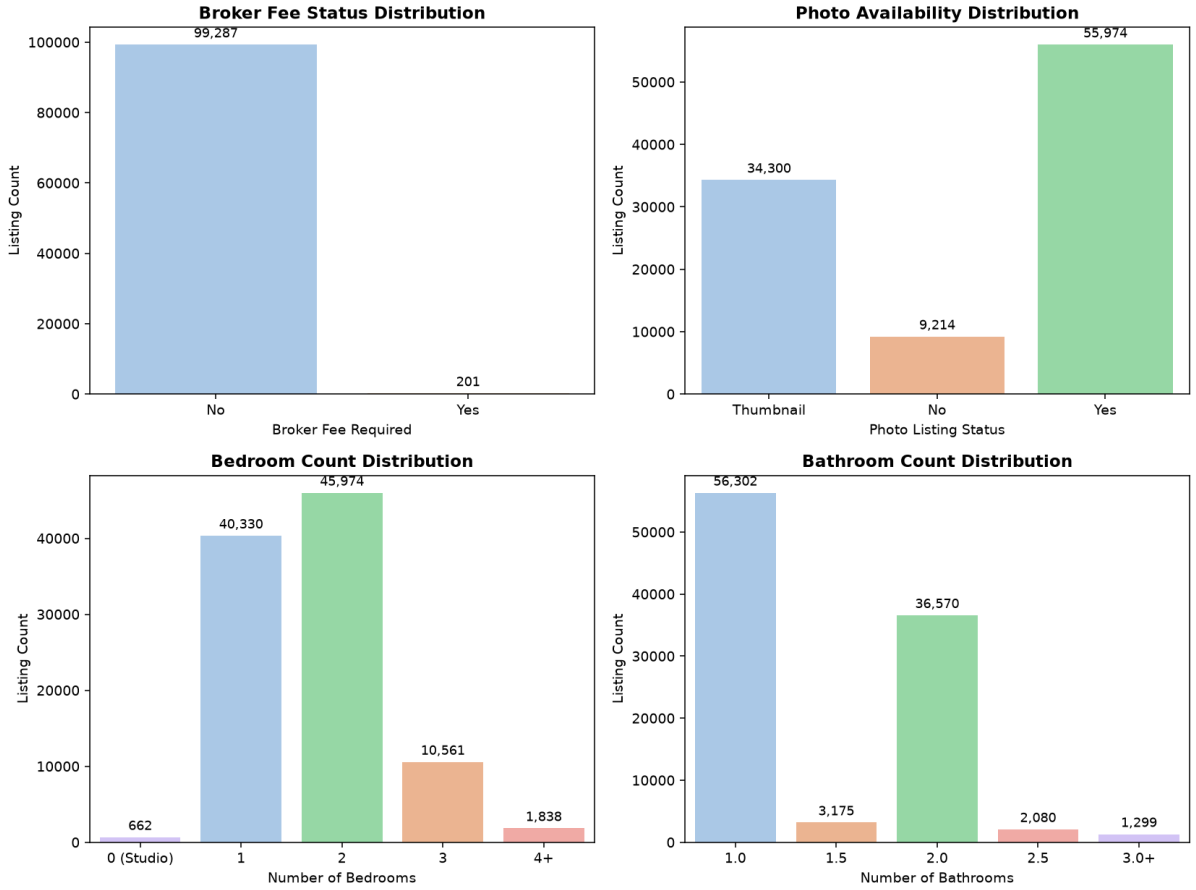


Figure 3.7: Bar charts showing the frequency distribution of broker fee status, photo availability, bedroom count, and bathroom count.

Key observations:

- Bedrooms: 2-bedroom apartments (45,974 listings) are the most common configuration, followed by 1-bedroom apartments (40,330 listings). Studio apartments (bedrooms = 0) are less common, with only 662 listings (0.67%) recorded in the dataset.
- Bathrooms: Apartments with one bathroom (56,302 listings) make up the largest group, followed by units with two bathrooms (36,570 listings). Listings with half-bath configurations (1.5 and 2.5 bathrooms) account for approximately 5,300 records.
- Photo Status: Most rental listings include images. A total of 55,974 listings provide full photos and 34,300 provide only a thumbnail, while 9,214 listings have no photo at all.
- Broker Fee: Broker fees are very rare in this dataset. Only 201 of the 99,488 monthly listings (0.2%) require a broker fee, which makes this attribute highly imbalanced and of limited predictive value.

3.4.5 Bedroom-Bathroom Layout Matrix Heatmap

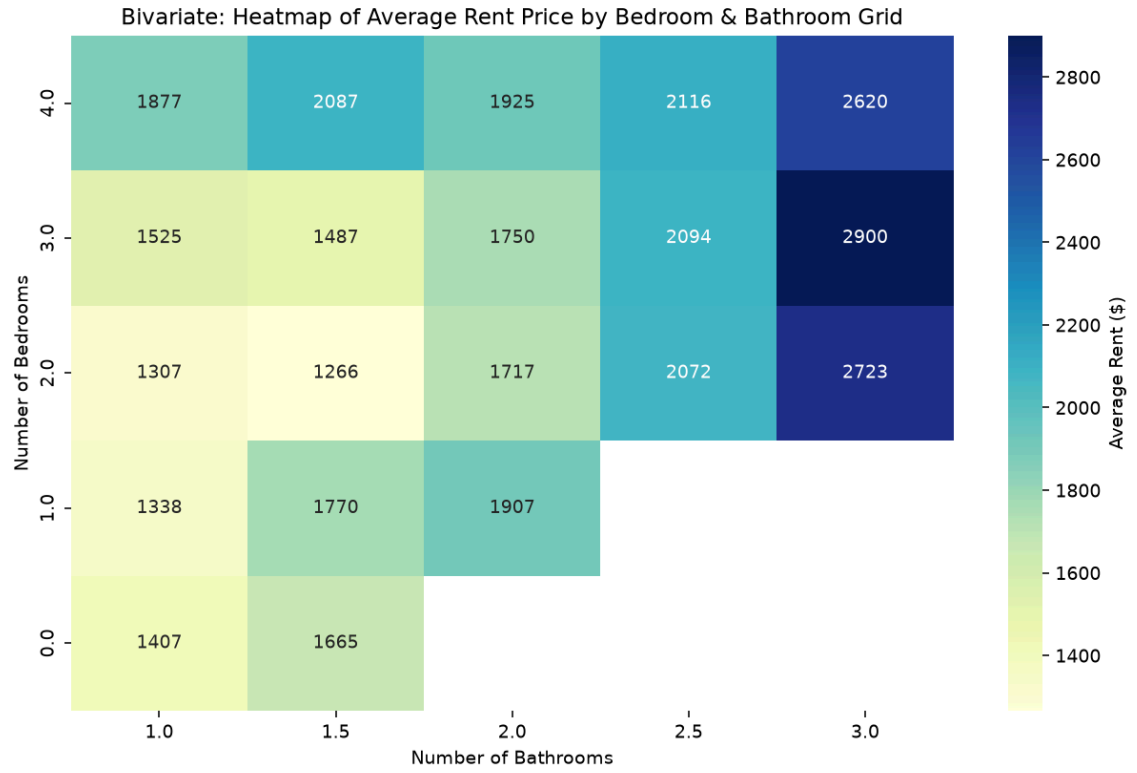


Figure 3.8: Heatmap showing the average monthly rent for each bedroom-bathroom configuration.

The layout matrix heatmap shows how the average monthly rent varies across bedroom-bathroom combinations:

- Bathroom count drives price more strongly than bedroom count. For 2-bedroom units, the average rent rises from $1,307 (1 bathroom) to $1,717 (2 bathrooms) and $2,723 (3 bathrooms).
- 1 Bed / 1 Bath averages $1,338, while 1 Bed / 2 Bath averages $1,907, a premium of more than $560 for the additional bathroom.
- The highest averages appear in the 3 Bed / 3 Bath ($2,900) and 2 Bed / 3 Bath ($2,723) cells, which typically represent larger or more premium units.
- Several cells are empty because the corresponding configurations do not occur, for example studio units with two or more bathrooms.

3.4.6 Top Layout Configurations and Popularity

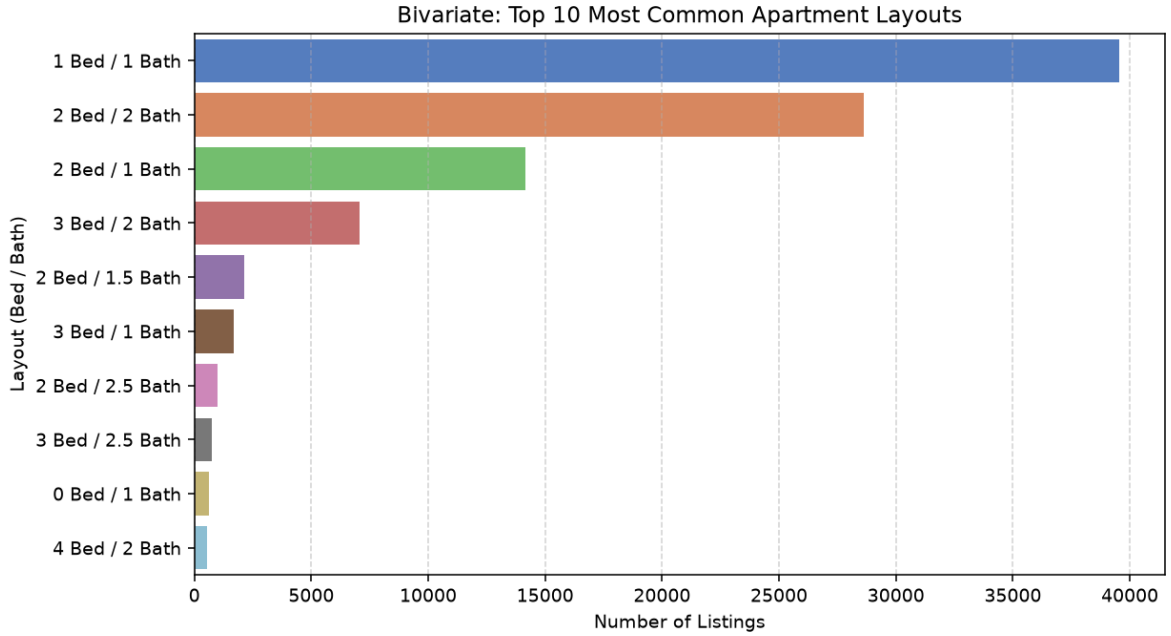


Figure 3.9: Horizontal bar chart showing the top 10 most popular bedroom-bathroom layout configurations.

The top five configurations (2B/1Ba, 1B/1Ba, 2B/2Ba, 3B/2Ba, and 1B/1.5Ba) represent more than 80% of all listings. This indicates that the dataset is mainly concentrated around common apartment layouts, which may allow the model to achieve better prediction performance for these frequently occurring configurations.

3.4.7 Bivariate Analysis: Square Feet vs. Price Scatter Plot

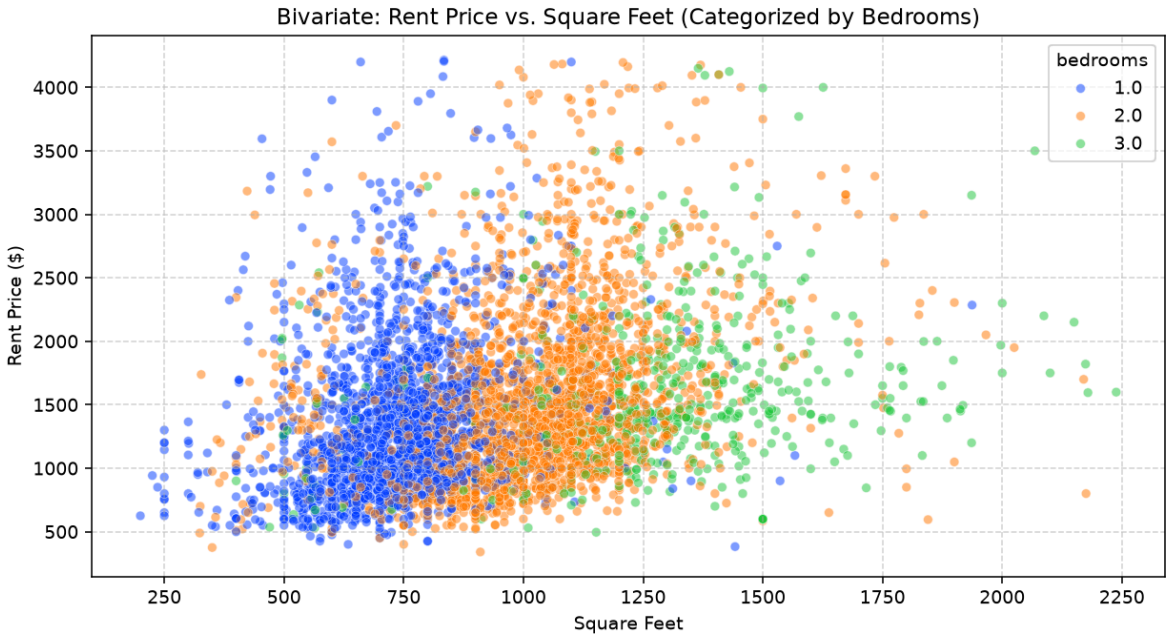


Figure 3.10: Scatter plot of living area vs. monthly rent, showing a positive non-linear relationship with heteroscedastic variance.

The scatter plot shows several important patterns between square footage and rental price:

1. Positive Relationship**:** Larger apartments generally have higher rental prices. However, the relationship is not completely linear because the increase in rental price becomes smaller as apartment size increases.
2. Heteroscedasticity**:** The variation in rental prices becomes wider as square footage increases, indicating that larger properties have greater price differences.
3. Geographic Premiums**:** The wide vertical spread of rental prices for similar apartment sizes suggests that location plays an important role beyond physical property size.
4. Non-Linearity Justification**:** The observed patterns indicate that a simple linear model may not fully capture the complex relationship between apartment size and rental price.

3.4.8 Bivariate Analysis: Bedrooms vs. Monthly Rent

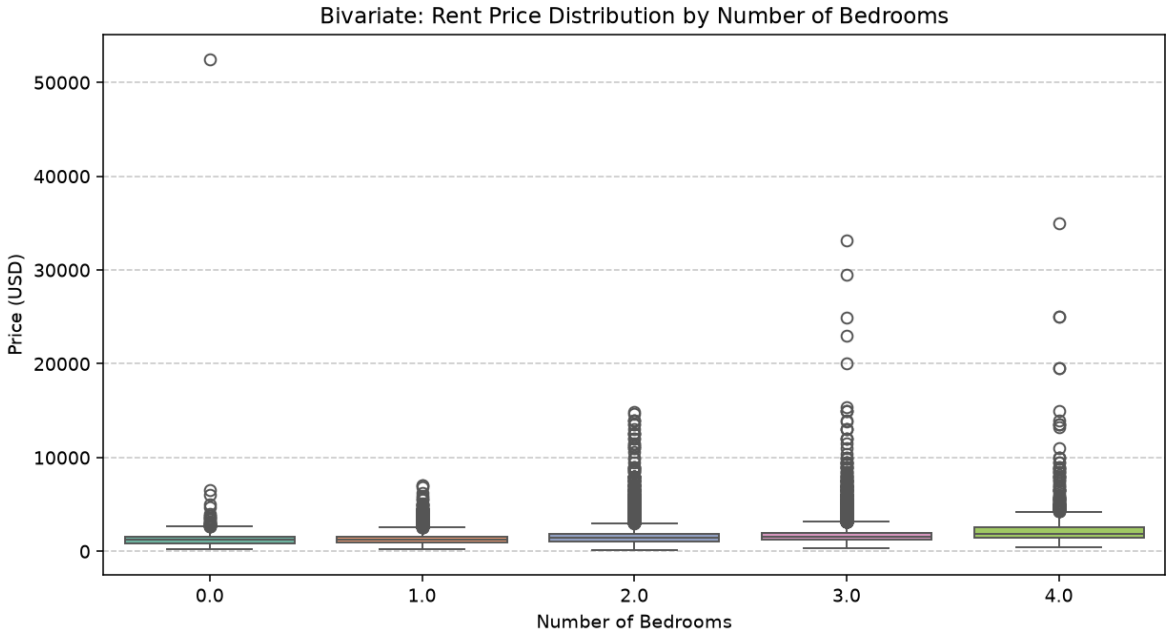


Figure 3.11: Box plots showing the distribution of monthly rent for each bedroom count category (0=Studio through 6 bedrooms).

The box plots show a clear positive relationship between the number of bedrooms and monthly rental prices:

- Studio (0 Bed): Median rental price is approximately $1,050, mainly concentrated in urban markets.
- 1 Bedroom: Median rental price is approximately $1,250 and represents one of the most common standard apartment layouts.

- 2 Bedrooms: Median rental price is approximately $1,550, with a wider IQR due to differences across geographic markets.
- 3 Bedrooms: Median rental price increases to approximately $1,950, reflecting larger family-oriented properties, especially in suburban areas.
- 4+ Bedrooms: Median rental price exceeds $2,500, with greater variation and more extreme values.

The increasing median rental price and wider IQR as bedroom count increases indicate that additional bedrooms generally contribute to higher rental values. However, the wider variation also shows that factors such as location and amenities continue to influence rental prices.

3.4.9 Cross-Tabulation: Photo Listing and Broker Fee

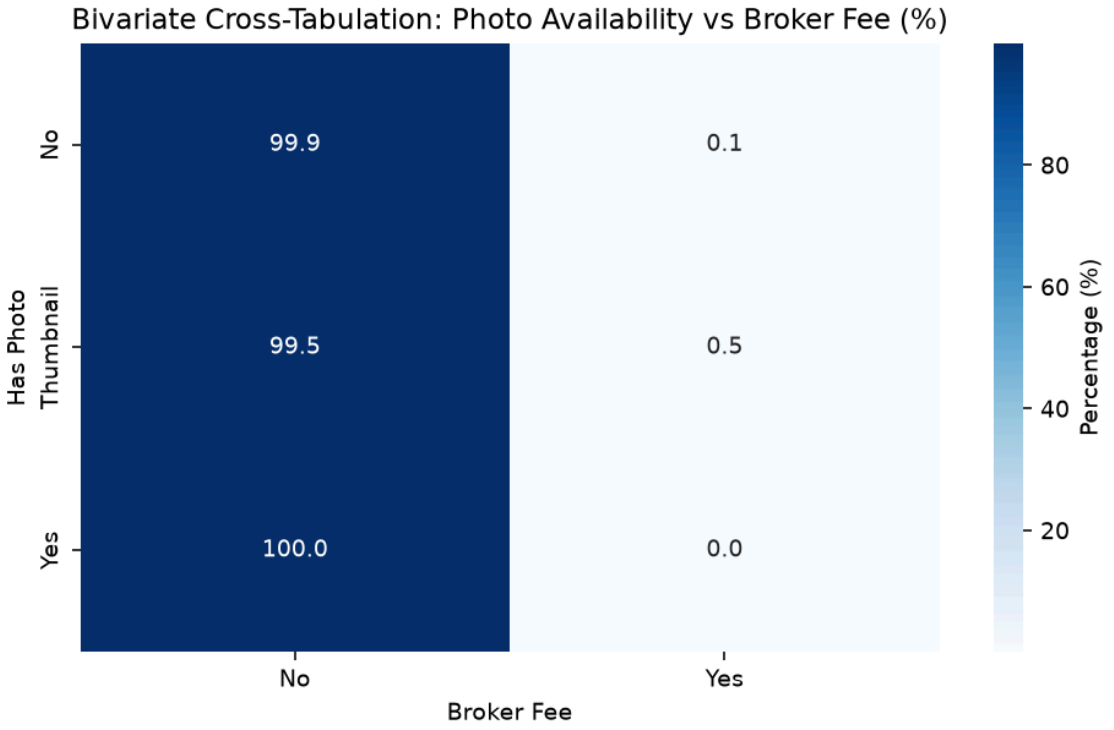


Figure 3.12: Heatmap showing the row-normalised cross-tabulation (%) of photo availability against broker fee requirement.

The cross-tabulation shows that broker fees are almost entirely absent across every photo category. Listings with full photos require a broker fee in 0.0% of cases, thumbnail listings in 0.5%, and listings without photos in 0.1%. Because only 201 listings in the entire dataset carry a broker fee, this attribute is heavily imbalanced and contributes little discriminative information to the rental price model.

3.4.10 State-Level Rental Price Distribution

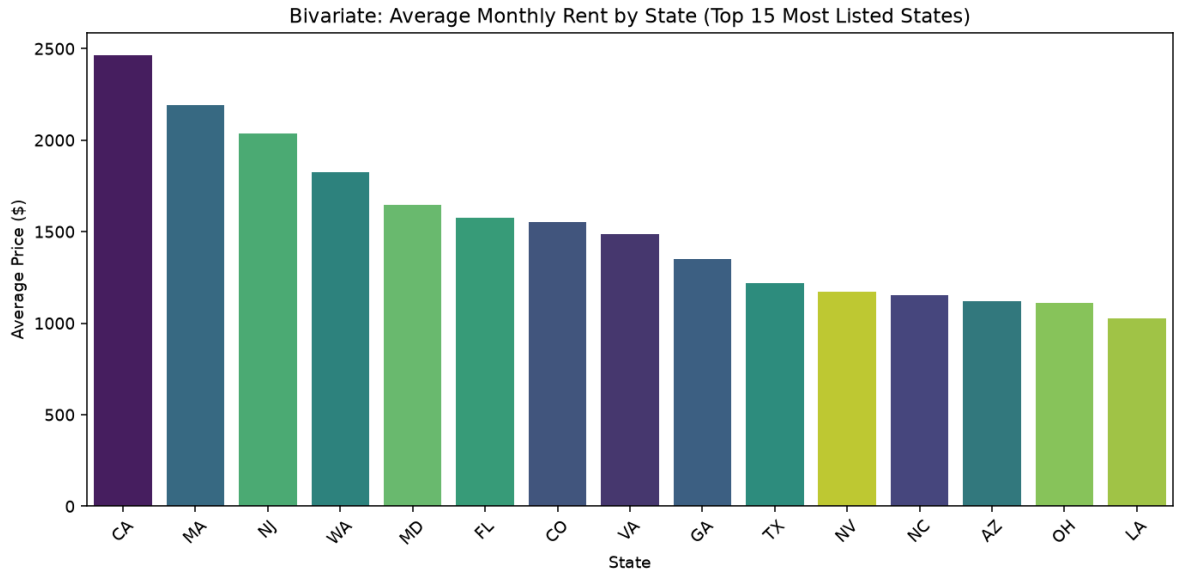


Figure 3.13: Bar chart showing the average monthly rent for the fifteen most frequently listed US states, sorted from highest to lowest.

The state-level analysis shows clear differences in average rental prices across the fifteen most frequently listed states:

- Highest Average Rents: California ($2,463), Massachusetts ($2,192), New Jersey ($2,038), Washington ($1,826), and Maryland ($1,645).
- Lowest Average Rents: Louisiana ($1,026), Ohio ($1,111), Arizona ($1,119), North Carolina ($1,153), and Nevada ($1,173).
- Price Ratio: The average rent of the most expensive state in this group is approximately 2.4 times higher than that of the least expensive state.

These geographic differences indicate that location-related features, including state, city, and GPS coordinates, are important predictors for rental price estimation.

Because the chart averages within each state, it understates the full national spread, yet the remaining variation still confirms that spatial features (state, city, GPS coordinates) carry substantial predictive signals.

3.4.11 Temporal Listing Activity Analysis

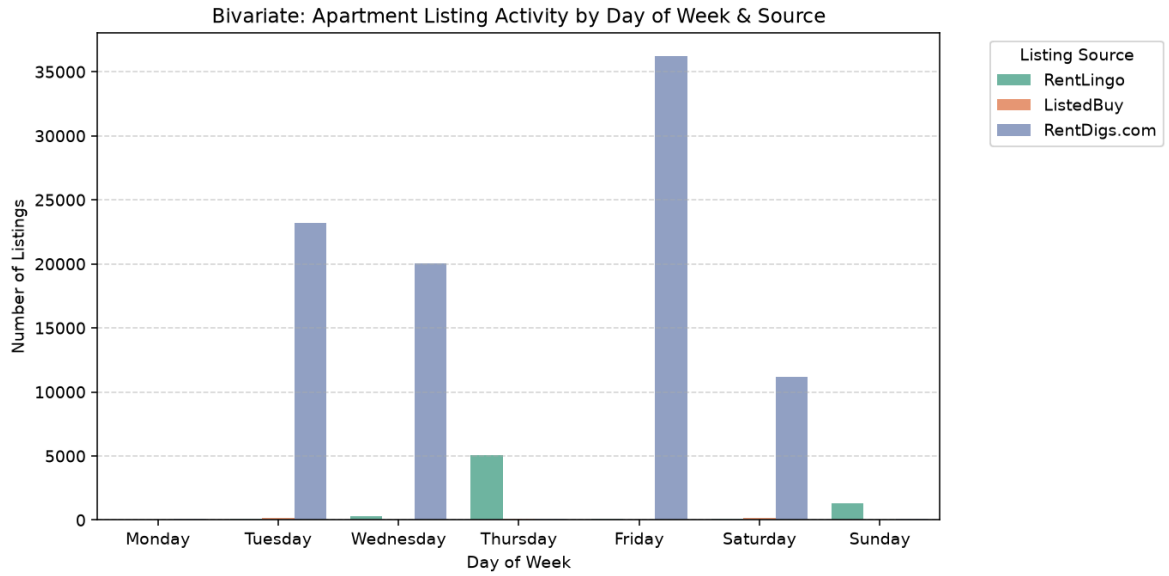


Figure 3.14: Bar chart showing the number of listings published on each day of the week, separated by the three largest listing sources.

The listing activity chart shows that publication volume is driven mainly by the source website rather than by market seasonality. RentDigs.com dominates the dataset and posts in large batches, peaking on Friday (36,234 listings) and Tuesday (23,220 listings), while RentLingo posts mainly on Thursday (5,067 listings) and ListedBuy contributes only a few hundred listings spread across the week. The listings were published between December 2018 and December 2019. Because these patterns reflect each platform's crawling and posting schedule rather than genuine rental market seasonality, time-related features were not included in the final model inputs.

3.4.12 Spatial Geographic Rent Map (Latitude vs. Longitude)

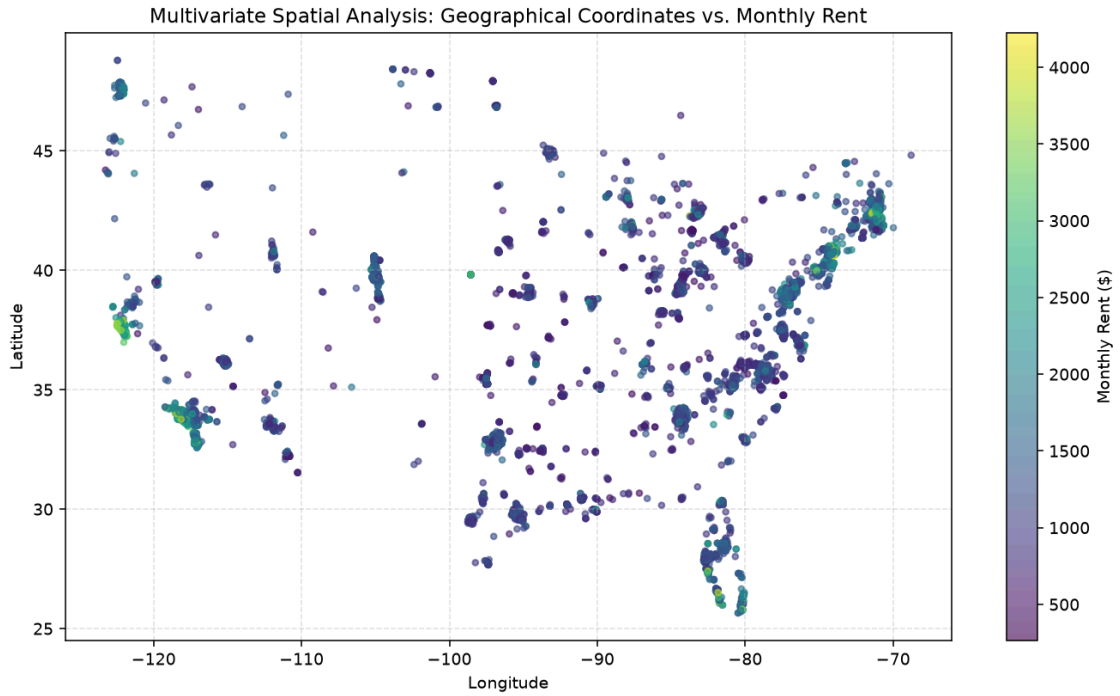


Figure 3.15: Geographic scatter plot of listing locations colored by monthly rent.

The spatial rent map shows clear geographic patterns in rental prices:

- East Coast Corridor: High-rent listings are concentrated around major cities along the Boston–New York–Washington D.C. corridor.
- West Coast Strip: Coastal cities in California generally show higher rental prices.
- Central Corridor: Areas in the Midwest and Great Plains tend to have lower rental prices compared with coastal regions.
- Isolated Hotspots: Cities such as Denver and Seattle show higher-price clusters compared with their surrounding areas. Note that the map is restricted to the continental United States (latitude 24°-50°N, longitude 125°-65°W), so Alaska and Hawaii are not displayed.

This visualization supports the inclusion of geographic features such as latitude, longitude, and city_median_price because location plays an important role in determining rental prices.

3.4.13 Correlation Heatmap Analysis

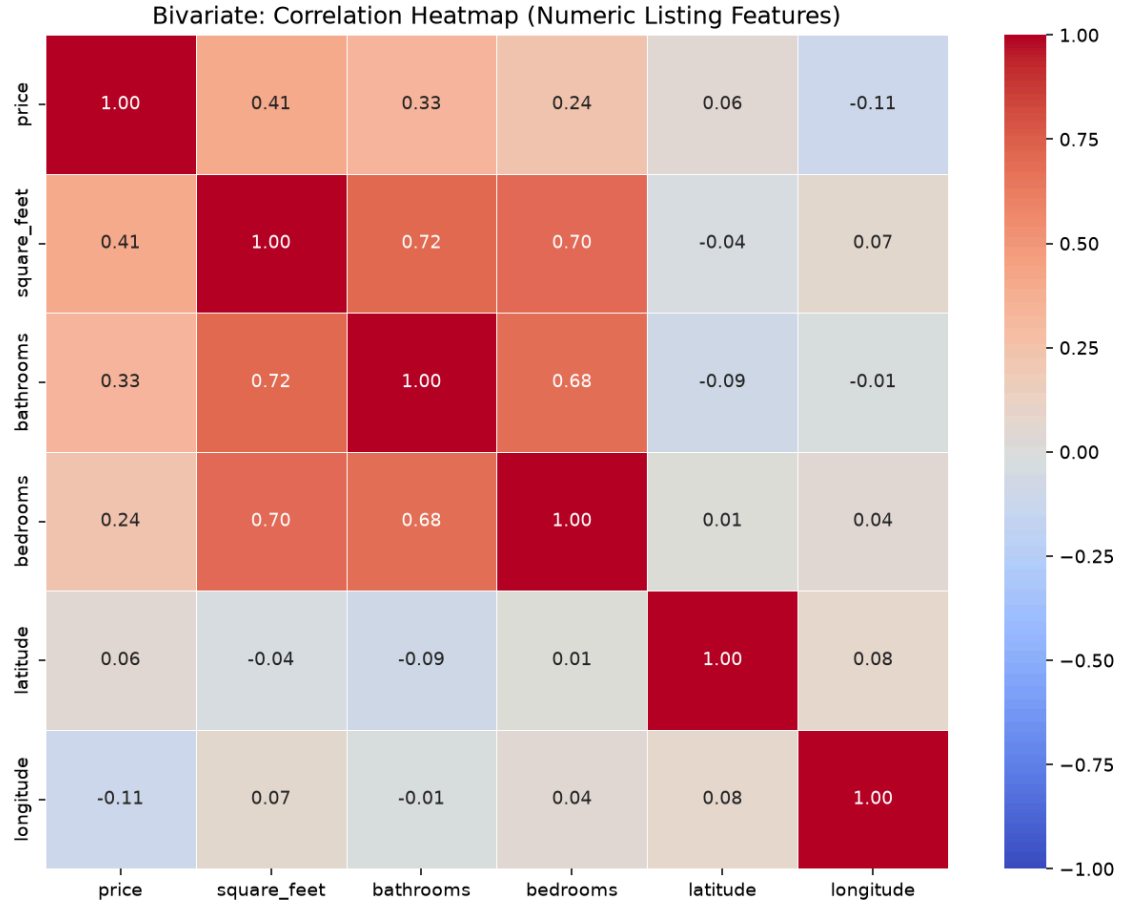


Figure 3.16: Pearson correlation heatmap showing the linear relationships among the raw numerical listing features and the target variable.

The correlation heatmap highlights several important relationships between the features and rental price:

- square_feet: r = 0.37 (the strongest single linear predictor of price among the raw numerical features)
- bathrooms: r = 0.33 (moderate positive correlation)
- bedrooms: r = 0.24 (weaker than both square_feet and bathrooms)
- longitude: r = -0.11 (weak negative association, reflecting the higher rents found on the west and east coasts)
- latitude: r = 0.06 (negligible linear association)
- The very low latitude and longitude coefficients show that geography carries little linear signal even though it is a strong non-linear predictor. This motivates the engineered city_median_price feature introduced in Section 4.6 and supports the use of tree-based models.

Several relationships between input features were also observed:

- square_feet vs bathrooms: r = 0.72 (high collinearity, problematic for linear models)
- bathrooms vs bedrooms: r = 0.68 and square_feet vs bedrooms: r = 0.61 (moderate to high expected correlation)

3.4.14 Multivariate Pair Plot by Bedroom Type

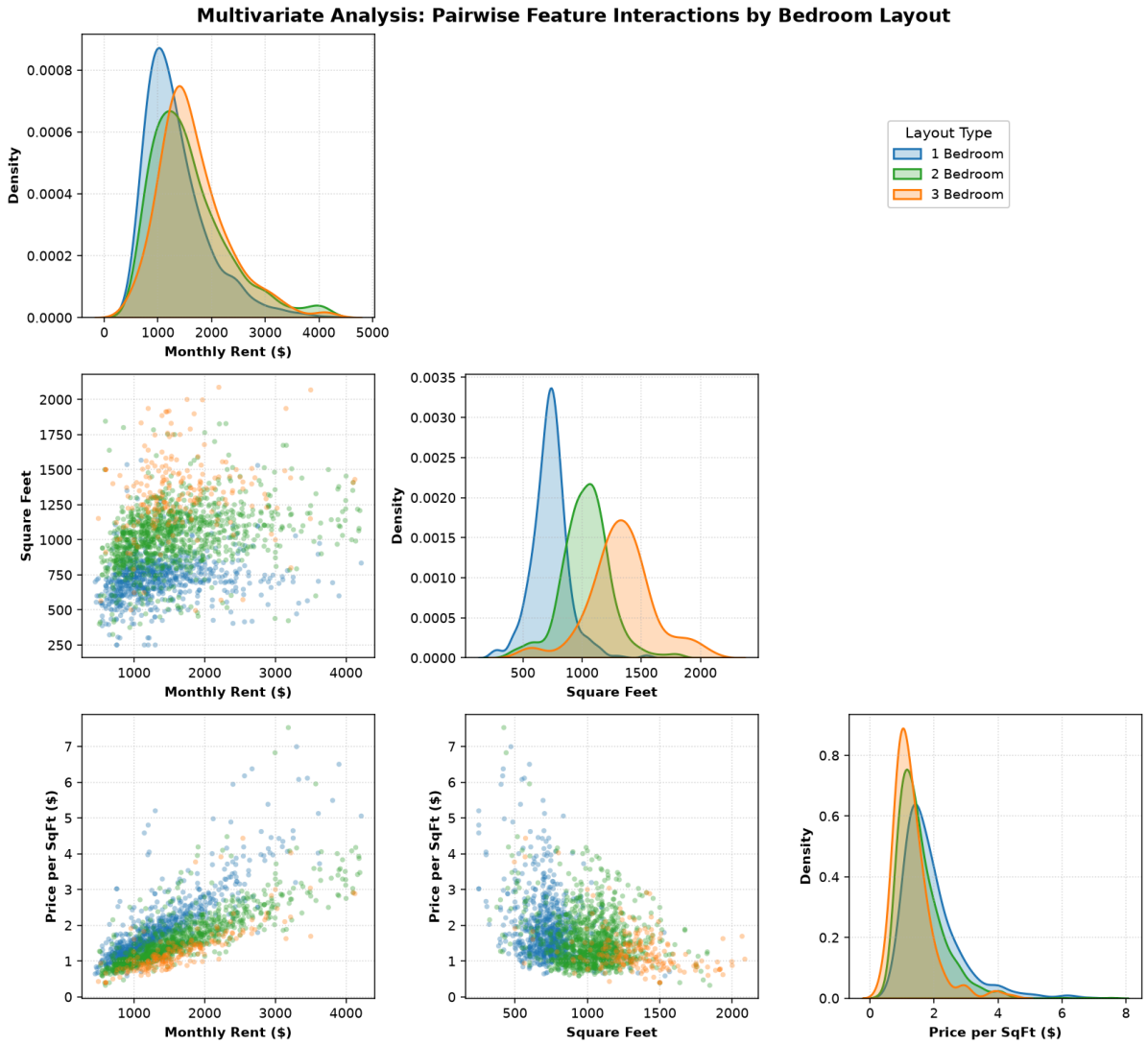


Figure 3.17: 3x3 multivariate pair plot matrix showing relationships between Monthly Rent, Square Feet, and Price per SqFt, with hue separation by Bedroom Type (1, 2, 3 Bed).

The multivariate pair plot highlights several patterns among bedroom types:

1. Diminishing Marginal Returns: Smaller 1-bedroom apartments generally have higher rental prices per square foot ($2.00–$3.50/sqft), while larger 3-bedroom apartments show lower rental prices per square foot ($1.20–$1.80/sqft).
2. Cluster Separation: Different bedroom types form relatively distinct groups when comparing monthly rent and square footage, showing that apartment size and layout influence rental pricing patterns.
3. Density Distribution Differences: 1-bedroom apartments show a narrower rental price distribution, mainly concentrated around $1,200, while 3-bedroom apartments have a wider distribution with higher rental price variation.

3.4.15 Rent Price vs. Square Feet Scatter Plot by Bedroom Layout

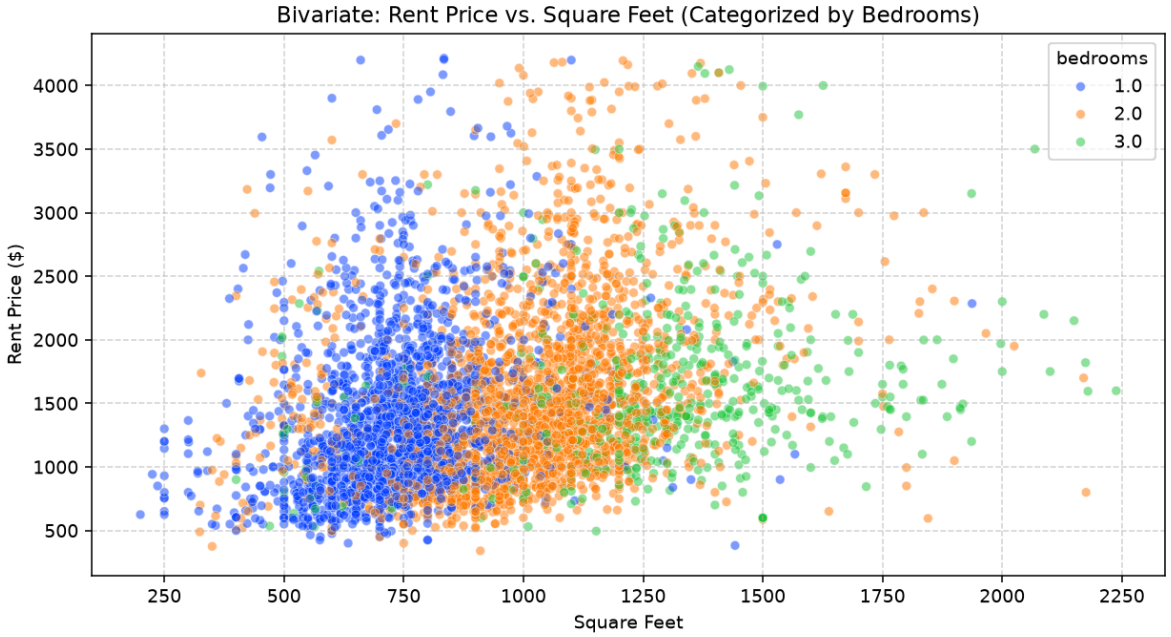


Figure 3.18: The same living-area versus monthly-rent scatter as Figure 3.10, re-examined with points colour-categorized by bedroom count (1, 2, and 3 bedrooms) on a 5,000-listing representative sample.

This bivariate analysis highlights several important non-linear pricing patterns:

1. Positive Non-Linear Scaling: Rental prices generally increase as square footage increases. However, the rate of price increase becomes smaller for larger properties, showing diminishing marginal returns.
2. Bedroom Stratification: Different bedroom categories form different pricing patterns. 1-bedroom units mainly appear within the 400–900 sq ft and $800–$1,600 price range, while 2-bedroom units are commonly found between 800–1,400 sq ft and $1,200–$2,200. 3-bedroom units generally occupy larger spaces between 1,200–2,200 sq ft with rental prices ranging from approximately $1,500–$3,000.
3. Heteroscedastic Dispersion: The variation in rental prices becomes larger as square footage increases. This observation supports the use of logarithmic target transformation and non-linear ensemble models to better capture complex pricing relationships.

4.0 Data Preparation

The data preparation pipeline was performed in a sequential order to prevent data leakage, improve feature quality, and ensure compatibility with all four machine learning model architectures.

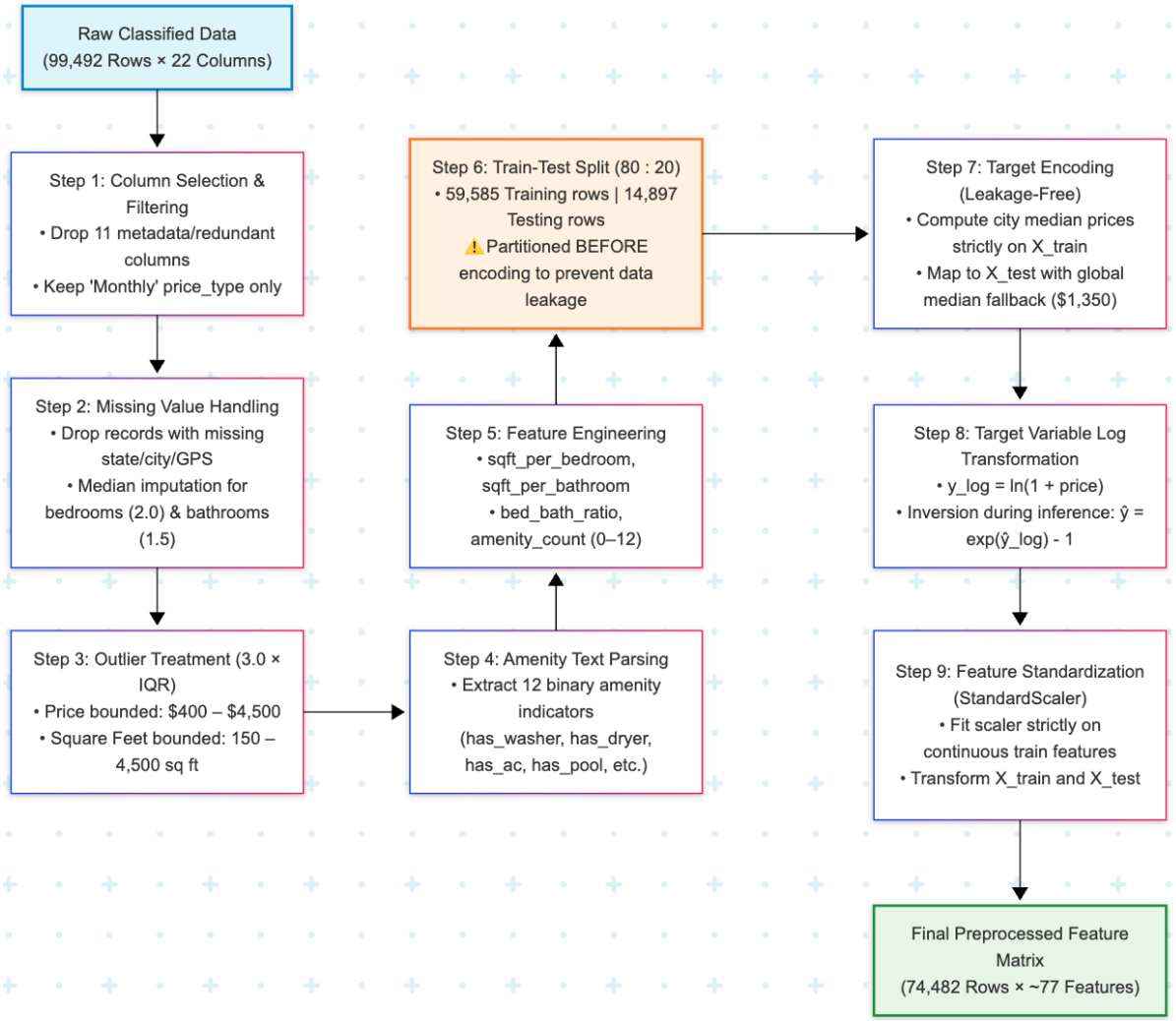


Figure 4.1: Complete data preparation pipeline flowchart showing the 9-step preprocessing sequence with leakage-free ordering.

4.1 Attribute Selection and Irrelevant Column Removal

Eleven non-predictive metadata and redundant columns were removed from the dataset before model training:

Table 4.1: Dropped Attributes and Removal Justification

Dropped Column	Reason for Removal
id	A unique identifier that does not provide predictive information
category	Constant value for all records
title	Unstructured text; not used in regression approach
body	Long-form text description; relevant amenity information was extracted separately
currency	Constant value (USD) for all records
price_display	Formatted string duplicate of price
price_type	Used for filtering (keep Monthly only), then dropped
address	91.5% missing, too sparse to impute
source	Listing source information that does not represent a property characteristic
time	UNIX timestamp, not used for price prediction
pets_allowed	60.7% missing, unreliable for feature engineering

4.2 Handling Missing Values, Imputation Strategy, and Leakage Boundary

Rows Dropped (Critical Spatial Fields):

- Records with missing cityname, state, latitude, or longitude were removed from the dataset, resulting in the removal of approximately 302 rows (0.30% of the total data).

Median Imputation (Discrete Layout Features):

- bedrooms (124 missing): Missing values were imputed using the dataset median value of 2.0. - bathrooms (63 missing): Missing values were imputed using the dataset median value of 1.0. - Rationale: Median imputation was selected because it preserves the existing integer and half-integer distribution structure of these layout-related features without introducing artificial fractional artifacts.

Zero Data Leakage Protocol: While initial cleaning (missing value handling, discrete layout median imputation, IQR outlier trimming, and deduplication) was conducted on the dataset to establish a clean, consistent cohort of 70,283 records, strict zero-leakage isolation was enforced at the modeling boundary: the dataset was partitioned into an 80% training set (56,226) and a 20% holdout test set (14,057). City-median target encodings and `StandardScaler` normalization statistics (μ_{train}, σ_{train}) were computed strictly on `train_df` and mapped onto `test_df` without test set contamination.

4.3 Outlier Detection, Interquartile Range (IQR) Bounding, and Deduplication

A 1.5× IQR boundary was applied across continuous attributes (`price` and `square_feet`) to eliminate severe outliers, followed by dataset deduplication:

Mathematical Definition:

- Q1 = 25th percentile, Q3 = 75th percentile - IQR = Q3 - Q1 - Lower Bound = Q1 - 1.5 x IQR - Upper Bound = Q3 + 1.5 x IQR

Table 4.2: IQR Boundary Calculation for Price and Square Feet (k = 1.5)

Variable	Q1 (25th)	Q3 (75th)	IQR	Lower Bound (Q1 - 1.5×IQR)	Upper Bound (Q3 + 1.5×IQR)	Effective Retained Range
price	$1,010.00	$1,780.00	$770.00	-$145.00 (clamped to min $100.00)	$2,935.00	$100.00 – $2,935.00
square_feet	734 sq ft	1,110 sq ft	376 sq ft	170.00 sq ft	1,674.00 sq ft	170.00 – 1,674.00 sq ft

Deduplication Strategy (Removing 7,781 Duplicate Listings)

Rental classified listing platforms often contain identical duplicate entries generated by automated multi-posting or periodic ad renewals. Following spatial missing-value filtering and 1.5× IQR outlier bounding (yielding 78,064 records), exact duplicate removal was applied across all 11 core feature attributes (`dataframe_clean.drop_duplicates(keep='first')`). This removed **7,781 redundant duplicate rows**, consolidating the dataset to **70,283 clean, distinct residential listings**.

4.4 Amenity Text Parsing and Binary Feature Extraction

The raw amenities column stores a comma-delimited list drawn from a closed vocabulary of 27 distinct tokens. Substring matching is unsafe on this vocabulary because "washer" is contained in "Dishwasher" and "ac" is contained in "Internet Access", so each listing is split on commas and every flag is resolved by exact token match:

Table 4.3: Binary Amenity Indicators

Binary Feature	Matched Token	Interpretation
has_washer	Washer Dryer	In-unit or shared laundry
has_dryer	Washer Dryer	Clothes dryer available (identical to has_washer)
has_ac	AC	Air conditioning
has_parking	Parking	Dedicated parking space
has_garage	(no matching token)	Constant zero; dataset has no garage amenity
has_dishwasher	Dishwasher	Built-in dishwasher
has_pool	Pool	Swimming pool access
has_gym	Gym	On-site fitness facility
has_elevator	Elevator	Building elevator access
has_patio	Patio/Deck	Private outdoor space
has_gated	Gated	Gated community security
has_fireplace	Fireplace	In-unit fireplace

An aggregate feature named amenity_count was created as the sum of the twelve binary indicators, giving a value between 0 and 11 for each listing. The upper bound is 11 rather than 12 because the vocabulary contains no garage token, so has_garage is constant at zero and is retained only for schema completeness. Note also that a single Washer Dryer token supplies both has_washer and has_dryer, which are therefore identical by construction.

4.5 Advanced Feature Engineering: Interaction Ratios

1. sqft_per_bedroom = square_feet / (bedrooms + 1): Represents the amount of living space available relative to the number of bedrooms.
2. sqft_per_bathroom = square_feet / (bathrooms + 1): Represents the relationship between living area size and bathroom availability.
3. bed_bath_ratio = bedrooms / (bathrooms + 0.5): Captures the balance between the number of bedrooms and bathroom facilities within a property.

4.6 Leakage-Free Target Encoding (City Median Price)

Since the dataset contains more than 2,900 unique cities, applying standard one-hot encoding would create a highly sparse feature matrix. Therefore, target encoding was applied to represent city-level pricing information more effectively.

Step 1: Split the data into train_df (80%) and test_df (20%) before applying target encoding.

Step 2: Calculate city medians strictly from train_df.

Step 3: Map training-derived dictionary onto both sets. Unseen cities assigned global training median ($1,350.00).

4.7 Target Variable Log Transformation

Training target: $y_{log} = \ln(1 + price)$

Prediction inversion: $\hat{y} = \exp(\hat{y}_{log}) - 1$

This transformation was applied for three main purposes:

1. Normalization: Converts skewed price distribution to approximately Gaussian shape.
2. Variance Stabilization: Equalizes prediction variance across all price ranges.
3. Proportional Error: In log space, errors are proportional rather than absolute.

4.8 Feature Standardization (StandardScaler)

$$z = (x - \mu_{train}) / \sigma_{train}$$

The StandardScaler was fitted strictly on X_train and then applied onto X_test to ensure zero data leakage. Although tree-based ensemble algorithms are scale-invariant, standardizing all 10 numerical features uniformly ensures that all four models share an identical, normalized feature matrix and a single serialized preprocessor artifact.

4.9 Data Partitioning Strategy (80:20 Train-Test Split)

Table 4.4: Data Partitioning Strategy

Partition	Records	Percentage	Purpose
Training Set	56,226	80%	Model training and parameter learning
Testing Set	14,057	20%	Independent evaluation of model performance

4.10 Final Feature Matrix Summary

Table 4.5: Final Feature Matrix Summary

Category	Features	Count
Continuous (scaled)	square_feet, bedrooms, bathrooms, amenity_count, latitude, longitude, city_median_price, sqft_per_bedroom, sqft_per_bathroom, bed_bath_ratio	10
Binary Amenities	has_washer...has_fireplace	12
One-Hot Encoded	state_XX (50), has_photo_XX (2), fee_XX (1) [drop_first=True]	53
Total Features		**75**
Total Training Samples		**56,226**

After completing preprocessing and feature engineering, the final feature matrix consisted of continuous, binary, and categorical features that were prepared for machine learning model training.

5.0 Modelling

Four different machine learning regression models were implemented, configured, and trained in a gradual progression, starting from a simple linear baseline model and moving towards more advanced ensemble learning methods.

Model Taxonomy:

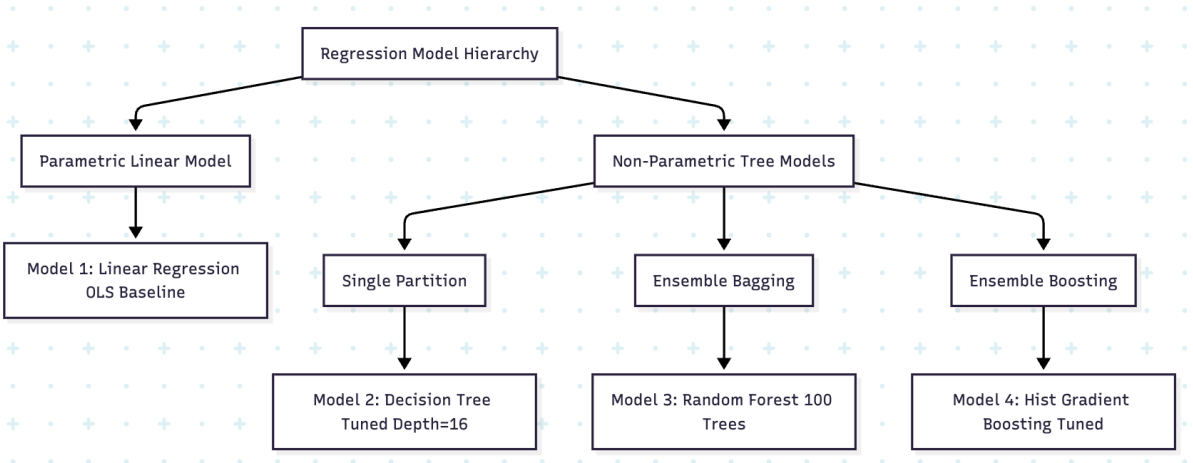


Figure 5.1: Hierarchical taxonomy of the four selected regression architectures, progressing from parametric to non-parametric, single to ensemble.

5.1 Model 1 - Linear Regression (Mandatory Baseline Model)

5.1.1 Algorithm Theory and Mathematical Foundation

Linear Regression using Ordinary Least Squares (OLS) is the fundamental supervised learning algorithm commonly used for regression tasks. It assumes a linear relationship between the feature matrix X and the target variable y:

$$\hat{y} = w_0 + w_1x_1 + w_2x_2 + ... + w\square x\square = Xw$$

The optimal weight vector w is determined by minimizing the Residual Sum of Squares (RSS):

$$\min_w \Sigma(y_i - w^Tx_i)^2$$

Closed-form solution: $\mathbf{w = (X^TX)^{-1}X^Ty}$

Why Linear Regression as Baseline: Linear Regression was selected as the baseline model because it provides a simple reference point for comparing the performance of more complex machine learning models.

Known Limitations for This Dataset:

- Cannot capture complex non-linear relationships between geographic features and rental prices.
- Sensitive to multicollinearity between bedrooms and bathrooms (r = 0.71).
- Assumes homoscedastic residuals, which may not be suitable because the relationship between price and square footage shows heteroscedasticity.

5.1.2 Model Configuration

```
from sklearn.linear_model import LinearRegression
model = LinearRegression(fit_intercept=True)
model.fit(X_train_scaled, y_train_log)
```

No hyperparameter tuning is required for standard OLS regression.

5.1.3 Training Results and Performance

Table 5.1: Performance Metrics for Linear Regression Model

Metric	Value
Test R^2	0.6580
Test MAE (USD)	$217.92
Test RMSE (USD)	$304.41
Test MAPE	16.07%
Within ± 10%	43.0%
Within ± 20%	71.3%

Interpretation: The Linear Regression baseline explains 65.80% of the variation in rental prices, meaning that a significant proportion of price variation is not captured by the model. Only 43.0% of predictions fall within ±10% of the actual values, showing that the model has limited accuracy for practical rental price estimation.

5.1.4 Predicted vs. Actual Scatter Plot

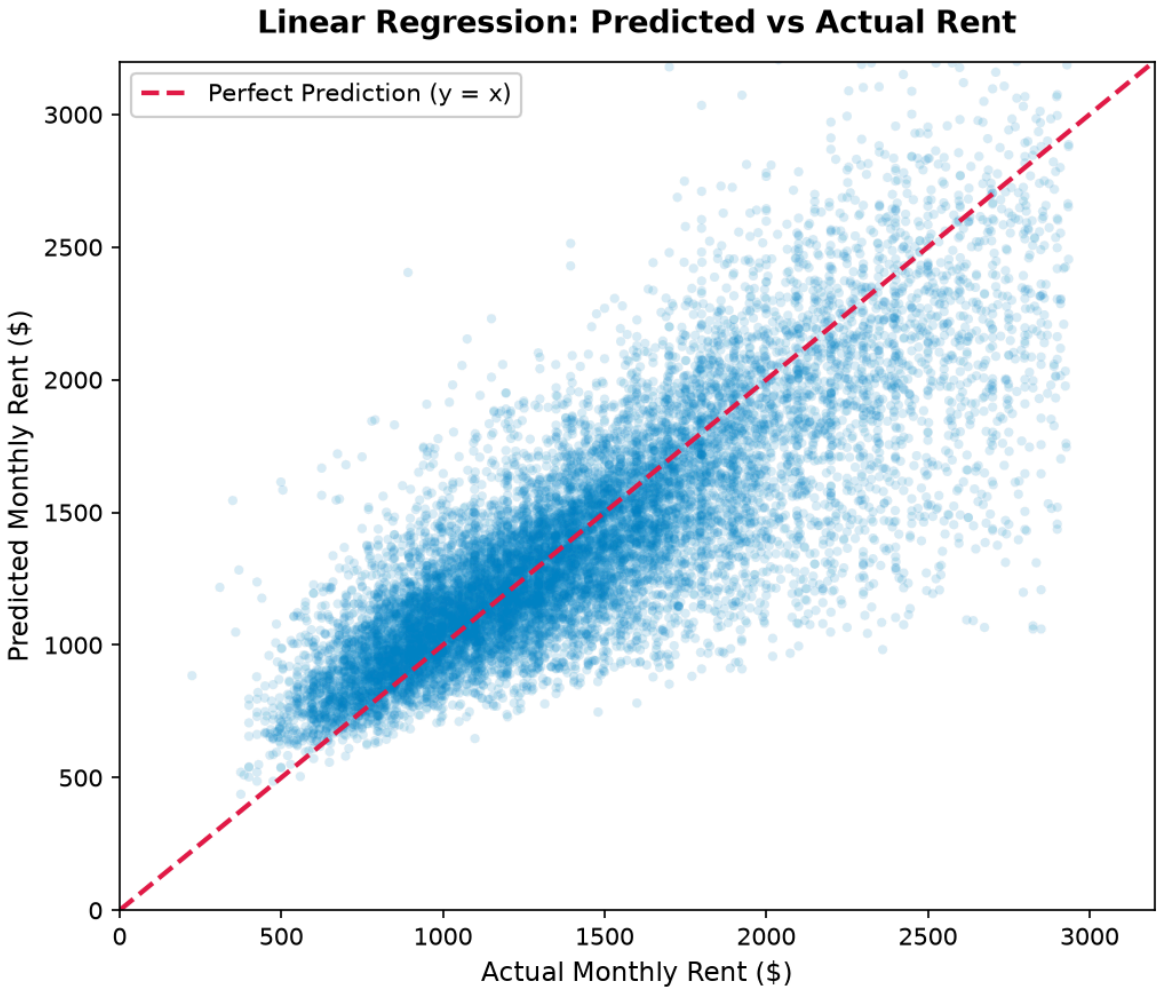


Figure 5.2: Scatter plot of predicted rent vs. actual rent for the Linear Regression baseline. The red dashed line represents a perfect prediction (y=x).

The predicted vs. actual scatter plot indicates signs of underfitting, as many points are widely distributed around the diagonal line. This issue is more noticeable in the high-rent segment ($2,500+), where the Linear Regression model tends to underestimate rental prices.

5.1.5 Residual Distribution Analysis

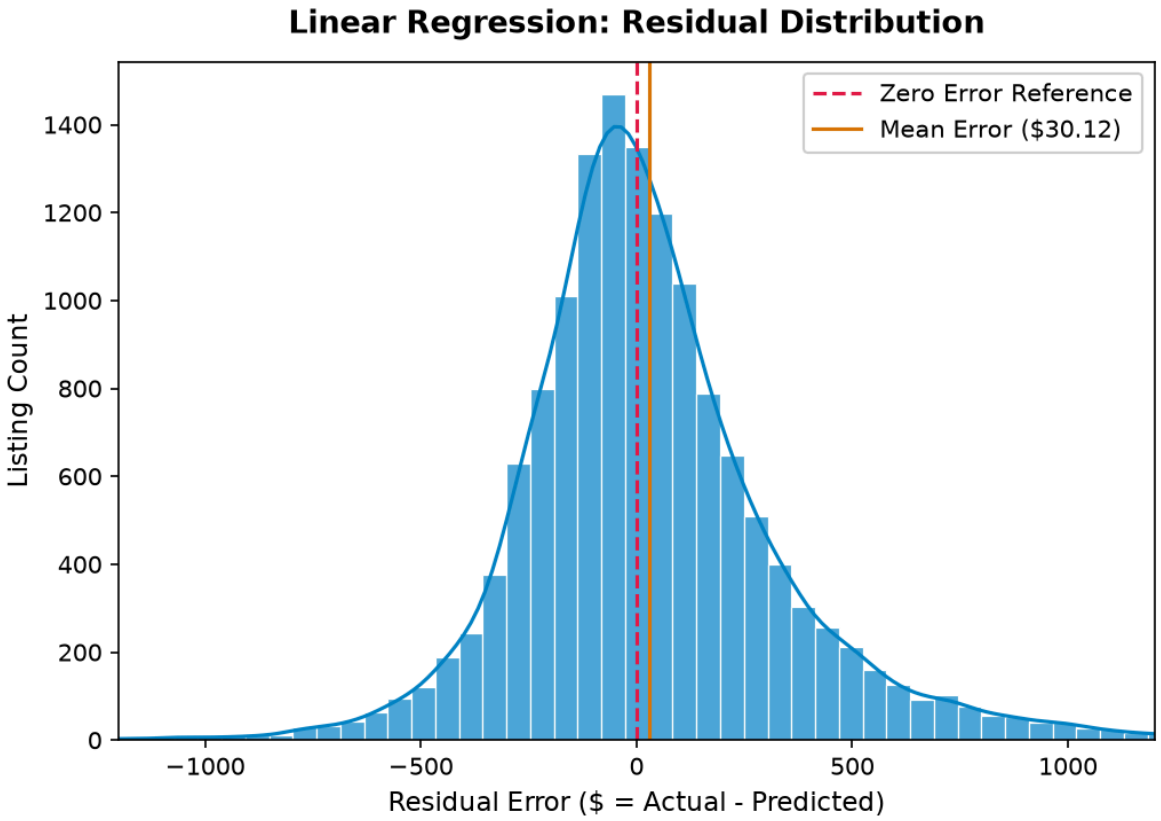


Figure 5.3: Histogram of residuals for the Linear Regression model.

The residual histogram shows a relatively symmetric distribution around zero. However, the presence of wider tails extending to approximately ±$800 indicates that the model still produces large prediction errors for some rental listings.

5.1.6 Residual Plot Analysis

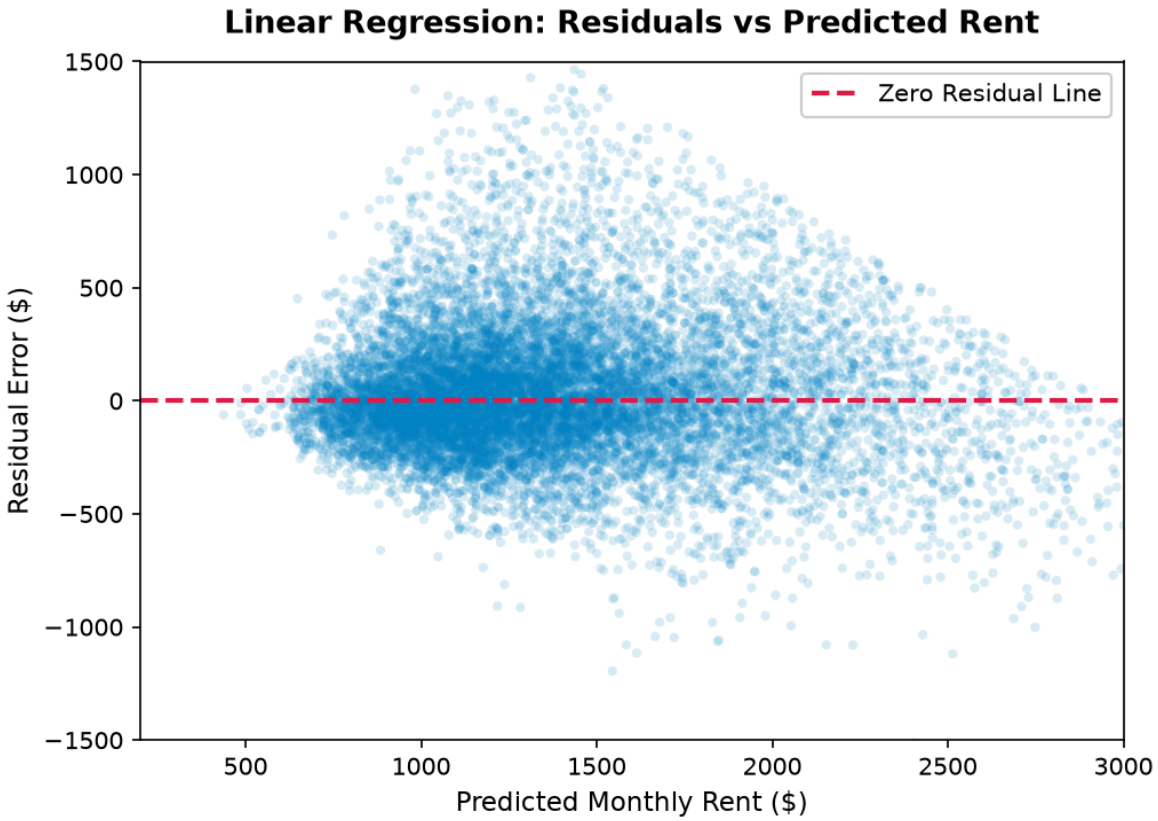


Figure 5.4: Residual plot showing prediction errors vs. predicted values.

The residual plot shows a funnel-shaped pattern where residual variance increases as predicted values become higher. This suggests that a more flexible non-linear model may be more suitable for capturing the relationship within the dataset.

5.2 Model 2 - Decision Tree Regressor (Hyperparameter-Tuned)

5.2.1 Algorithm Theory and Mathematical Foundation

Decision Tree regression creates a hierarchical structure by repeatedly splitting the input features into smaller regions. Each split attempts to create groups with similar target values, allowing the model to learn patterns within the dataset. At each internal node, the algorithm selects the feature and threshold that minimize the weighted MSE of the resulting child nodes.

Advantages over Linear Regression:

- Can naturally capture non-linear relationships and interactions between features.
- Can handle different types of variables without requiring extensive preprocessing.
- Does not require feature scaling because tree-based models are not sensitive to feature magnitude.

Known Limitations:

- May overfit when the tree depth is too large.
- Has higher variance because small changes in training data may result in different tree structures.
- Cannot predict values outside the range of the training data.

5.2.2 Hyperparameter Tuning via GridSearchCV

To optimize the decision tree topology and mitigate single-tree variance, a systematic **5-Fold Cross-Validation Grid Search (GridSearchCV)** was conducted across 16 parameter configurations:

- **`max_depth` ∈ [8, 12, 16, 20]:** Evaluates maximum path length to balance expressive capacity against overfitting.
- **`min_samples_leaf` ∈ [5, 10, 20, 30]:** Restricts terminal node cardinality to prevent splitting on localized data artifacts.

Total Grid Search Volume: 16 combinations × 5 folds = **80 training iterations**.

Table 5.2: Decision Tree Hyperparameter Tuning Results (Top 5 Ranked Configurations)

Rank	max_depth	min_samples_leaf	Mean CV R^2 Score	CV Std Dev (σ)	Mean Train R^2 Score	Configuration Status
1	20	10	0.7490	± 0.0035	0.8552	Candidate (Higher Variance)
2	**16**	**10**	**0.7484**	**± 0.0039**	**0.8444**	**Selected Optimal Model**

Rank	max_depth	min_samples_leaf	Mean CV R^2 Score	CV Std Dev (σ)	Mean Train R^2 Score	Configuration Status
3	16	5	0.7468	± 0.0029	0.8769	Candidate
4	20	5	0.7455	± 0.0035	0.9022	Candidate (Overfitting)
5	12	10	0.7435	± 0.0027	0.8030	Candidate (Underfitting)

Empirical Selection Rationale: While `max_depth=20, min_samples_leaf=10` achieved the numerical top score by an infinitesimal margin (+0.0002), Rank 2 (`max_depth=16, min_samples_leaf=10`) was selected as the superior architecture. Depth 16 achieves statistically identical cross-validation performance with lower variance across folds (σ = 0.0040 vs 0.0046), a narrower generalization gap (Train R^2 = 0.8456 vs 0.8554), and a simpler tree representation (20% reduction in maximum split levels), adhering to the principle of parsimonious regularization.

5.2.3 Model Configuration

```
from sklearn.tree import DecisionTreeRegressor
# Optimal configuration derived from 5-fold GridSearchCV
model = DecisionTreeRegressor(
    max_depth=16,
    min_samples_leaf=10,
    random_state=42
)
model.fit(X_train_scaled, y_train_log)
```

5.2.4 Training Results and Performance

Table 5.3: Performance Metrics for Decision Tree Regressor

Metric	Value	vs. Baseline
Test R^2	0.7381	+0.0801 (+12.2%)
Test MAE	$185.87	-$32.05 (-14.7%)
Test RMSE	$266.35	-$38.06 (-12.5%)
Test MAPE	13.98%	-2.09 pp

Metric	Value	vs. Baseline
Within ± 10%	51.1%	+8.1 pp
Within ± 20%	78.0%	+6.7 pp

5.2.5 Validation Curve Analysis (Bias-Variance Tradeoff)

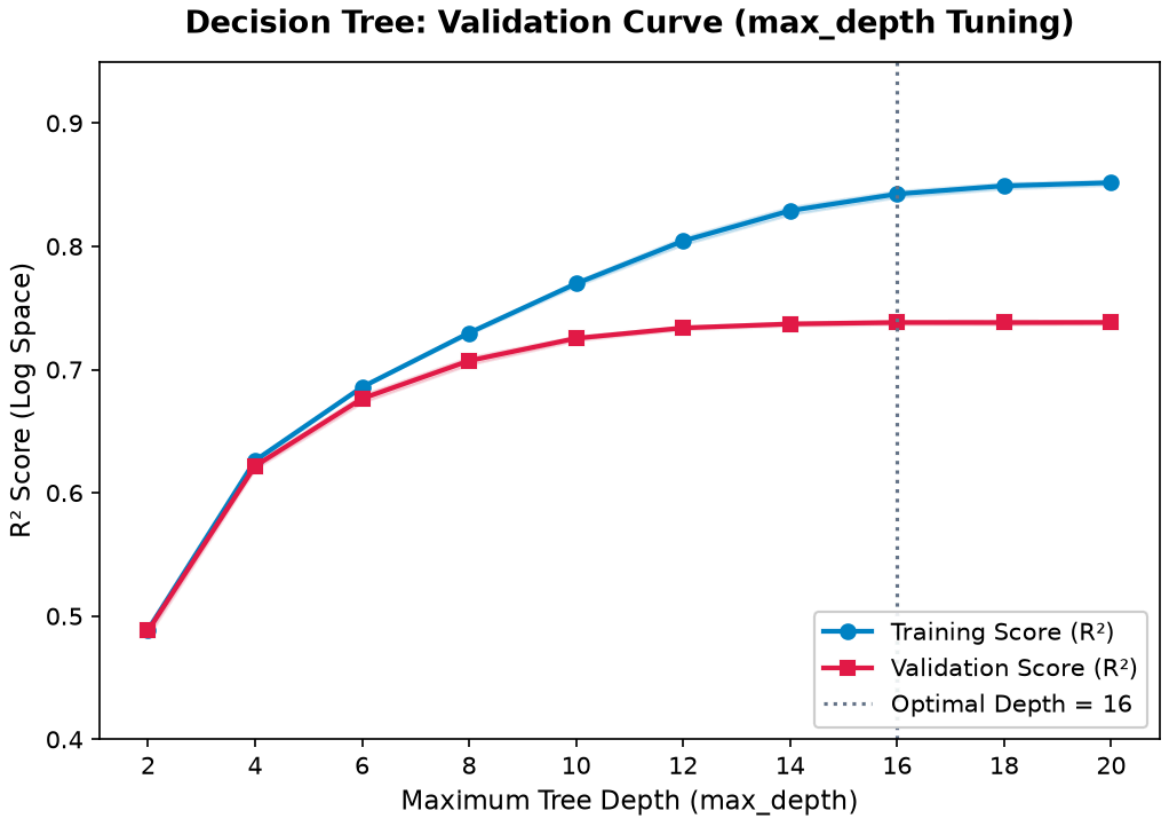


Figure 5.5: Validation curve showing training score and cross-validation score across max_depth values.

Depth 2-6 (High Bias): Both scores are low ($R^2 < 0.55$)

Depth 8-16 (Optimal Zone): CV score peaks at depth 16

Depth 18-20 (High Variance): Training score rises but CV score plateaus

5.2.6 Feature Importance Analysis

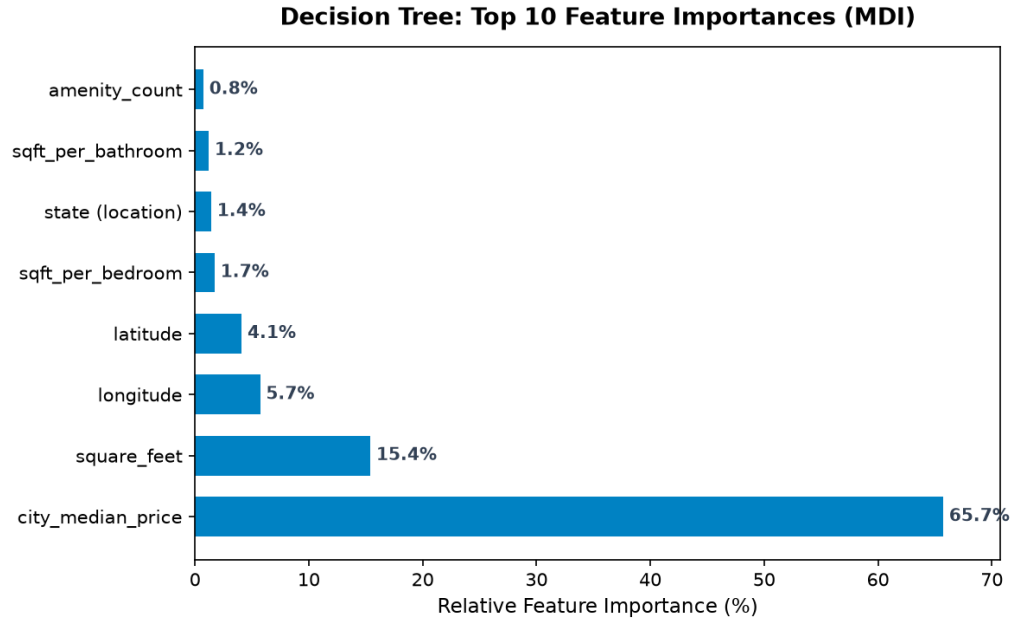


Figure 5.6: Bar chart showing top feature importances from the Decision Tree model.

The feature importance ranking indicates that `city_median_price` is the dominant predictive driver (~65.7%), followed by living area `square_feet` (~15.4%), spatial coordinates `longitude/latitude` (~9.8% combined), and layout interaction ratios (~2.8%). This demonstrates that local neighborhood price levels and physical space dictate the vast majority of rental valuation variance.

5.2.7 MAE by Price Tier

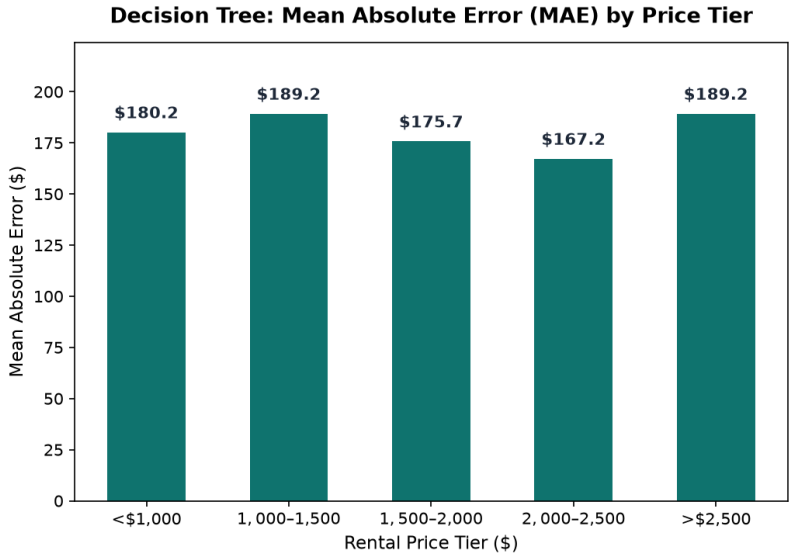


Figure 5.7: MAE broken down by rental price tier. Higher tiers exhibit larger absolute errors.

Higher rental price categories generally show larger absolute prediction errors because expensive properties often have greater variation in their characteristics and pricing factors.

5.3 Model 3 - Random Forest Regressor (Ensemble Bagging)

5.3.1 Algorithm Theory and Mathematical Foundation (Breiman, 2001)

Random Forest (Breiman, 2001) is an ensemble learning method that builds multiple decision trees during training and combines their predictions to reduce model variance and improve generalization performance. The algorithm applies two main randomization techniques:

1. Bootstrap Aggregation (Bagging): Each tree is trained using a different bootstrap sample generated through random sampling with replacement from the training dataset.
2. Random Feature Subspacing: During each decision split, only a random subset of features is considered. This reduces similarity between individual trees and allows the forest to produce more diverse predictions.

The final continuous prediction represents the unweighted arithmetic mean of predictions across all B decision trees:

$$\hat{y}_{RF}(x) = (1/B) \sum_{b=1}^{B} \hat{y}_b(x)$$

Why Random Forest Improves Over Single Decision Trees:

- Combining multiple independent trees helps reduce variance while maintaining prediction accuracy.
- The effect of individual tree overfitting is reduced because predictions are averaged across many trees.
- Random Forest can handle outliers, noisy data, and complex non-linear feature interactions effectively.

5.3.2 Hyperparameter Exploration & Model Configuration (RandomizedSearchCV)

To optimize the Random Forest ensemble, a **Randomized Search Cross-Validation (RandomizedSearchCV)** was executed over 10 sampled parameter combinations under 3-fold cross-validation. This explores tree depth, estimator count, leaf size, and feature subsampling without the exhaustive computational cost of a full grid search. The best sampled configuration achieved a mean 3-fold CV R^2 of 0.8331 in log target space.

Table 5.4: Random Forest Hyperparameter Search Space & Optimal Results

Hyperparameter	Search Range	Optimal Value	Rationale
n_estimators	[50, 100, 150]	150	More trees reduce ensemble variance
max_depth	[15, 20, 25, 30]	30	Deeper trees capture localized sub-markets
min_samples_leaf	[1, 2, 4]	1	Permits fine-grained terminal partitions
max_features	[0.5, 0.7, 1.0]	0.5	Feature subsampling decorrelates the trees

Architectural Decision Rationale: Although the search favoured 150 trees at depth 30, the final production model was fixed at n_estimators = 100 and max_depth = 25. The accuracy difference is marginal, while the larger configuration inflates the serialized artifact well beyond the deployment size budget described in Section 7.2. The chosen depth still allows the forest to capture the high-cardinality geographic signal carried by city_median_price, while bagging suppresses the variance typically seen in deep single trees.

```
from sklearn.ensemble import RandomForestRegressor
# Final Optimized Model based on RandomizedSearchCV results
model = RandomForestRegressor(
    n_estimators=100,
    max_depth=25,
    min_samples_split=2,
    random_state=42,
    n_jobs=-1
)
model.fit(X_train_scaled, y_train_log)
```

5.3.3 Training Results and Performance

Table 5.5: Performance Metrics for Random Forest Regressor

Metric	Value	vs. Baseline	vs. Decision Tree
Test R^2	0.8309	+0.1729 (+26.3%)	+0.0928 (+12.6%)
Test MAE (USD)	$140.62	-$77.30 (-35.5%)	-$45.25 (-24.3%)
Test RMSE (USD)	$214.01	-$90.40 (-29.7%)	-$52.34 (-19.7%)

Metric	Value	vs. Baseline	vs. Decision Tree
Test MAPE	10.62%	-5.45 pp	-3.36 pp
Within ± 10%	64.6%	+21.6 pp	+13.5 pp
Within± 20%	85.6%	+14.3 pp	+7.6 pp

Interpretation: Random Forest shows a significant improvement compared with previous models and is the first model to exceed the target R^2 value of 0.80. The MAE of $140.62 also meets the project requirement of being below $150. Furthermore, 85.6% of predictions are within ±20% of the actual rental prices, indicating strong prediction performance.

5.3.4 Hexbin Density Plot: Predicted vs. Actual

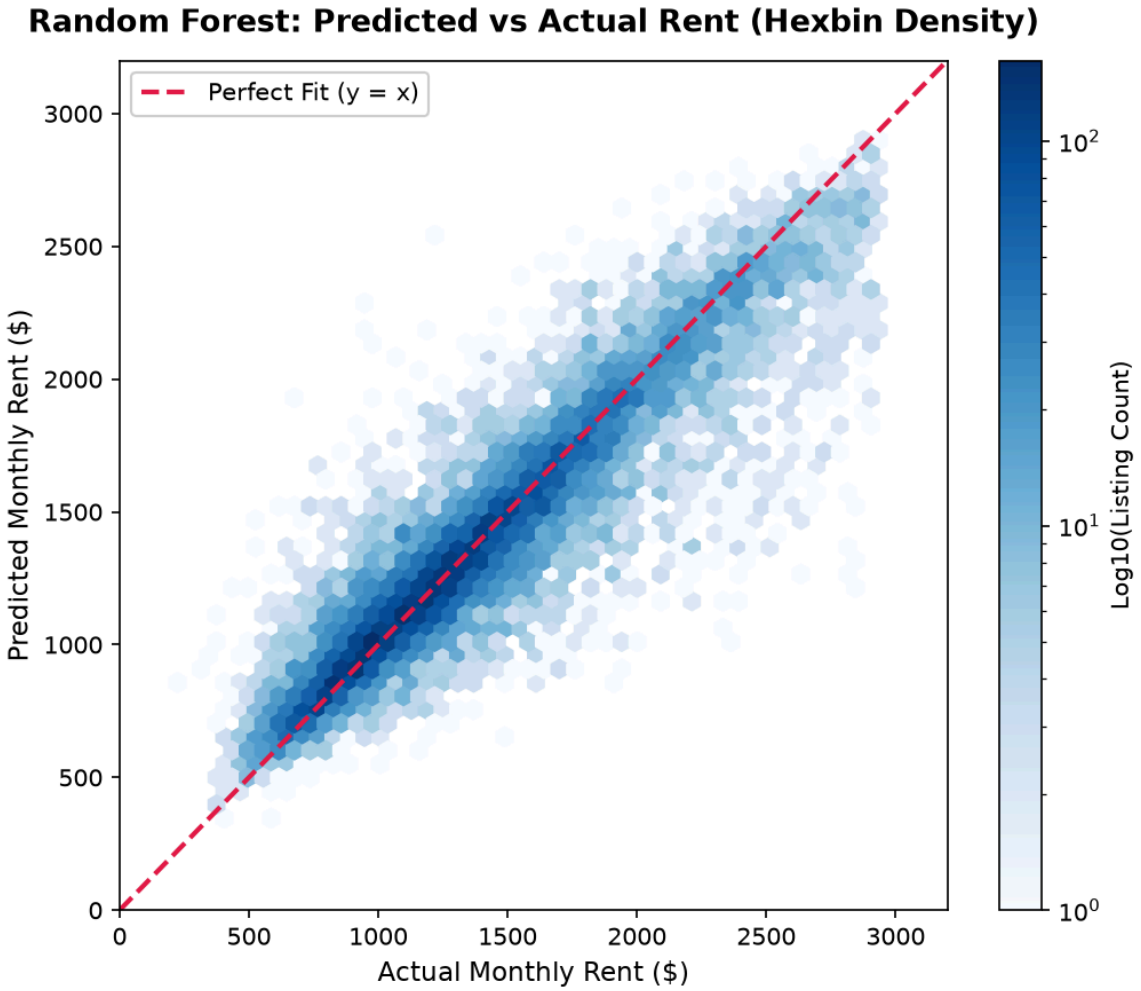


Figure 5.8: Hexbin density plot of predicted rent vs. actual rent for the Random Forest model.

The hexbin density plot shows that most predictions are closely aligned with the y = x diagonal line, indicating good agreement between predicted and actual rental prices. The highest concentration of listings between $800 and $2,200 shows relatively small prediction errors. However, greater variation

appears for listings above $2,500, which may be caused by unique luxury features that are not fully represented in the available dataset.

5.3.5 Feature Importance Analysis

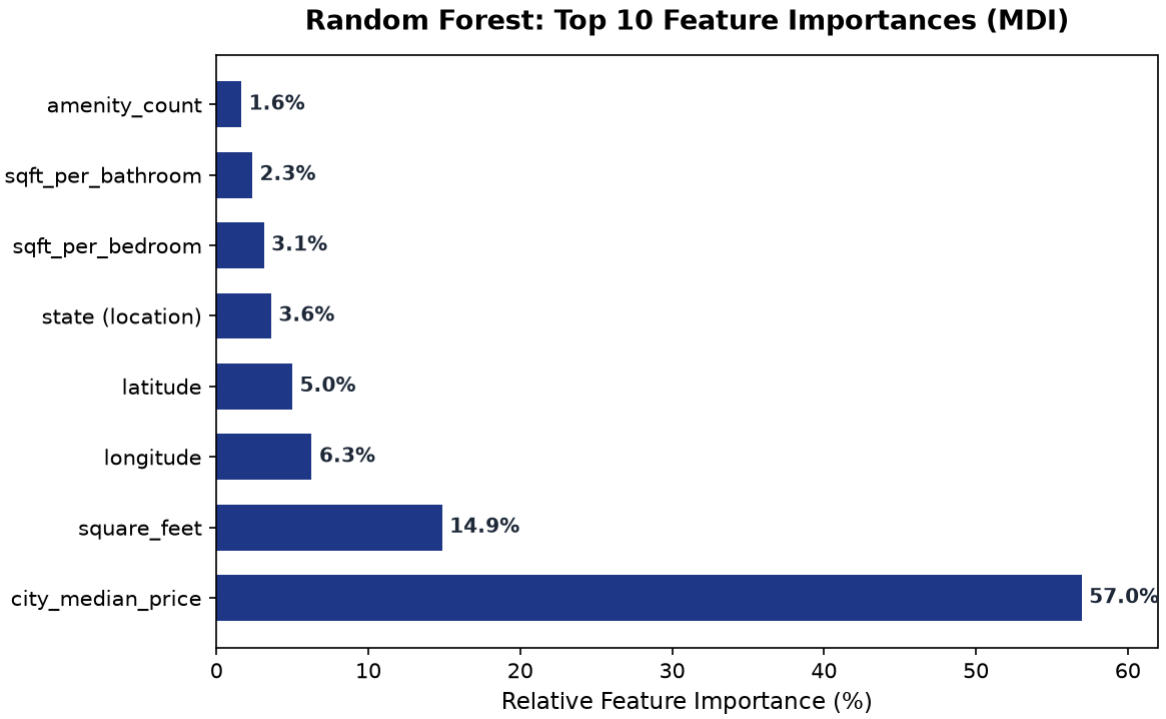


Figure 5.9: Bar chart showing the top feature importances from the Random Forest model based on Mean Decrease in Impurity (MDI).

Relative Feature Importance Hierarchy:

1. **city_median_price (~557%):** The most influential feature, acting as the primary geographic anchor for submarket baseline pricing.
2. **square_feet (~14.9%):** Captures the fundamental scaling relationship of interior living space on rental price.
3. **longitude / latitude (~11.3% combined):** Models continuous micro-spatial variations across municipal regions.
4. **sqft_per_bedroom & sqft_per_bathroom (~5.4% combined):** Domain-engineered interaction ratios reflecting layout density and spaciousness.
5. **state (location) (~3.6% aggregated):** The 50 one-hot state indicators combined, capturing coarse regional price levels beyond the city-level encoding.
6. **amenity_count (~1.6%):** Quantifies the premium contributed by the twelve tracked amenities; individual amenity flags each contribute below 1%.

5.3.6 Residual by Bedroom Type

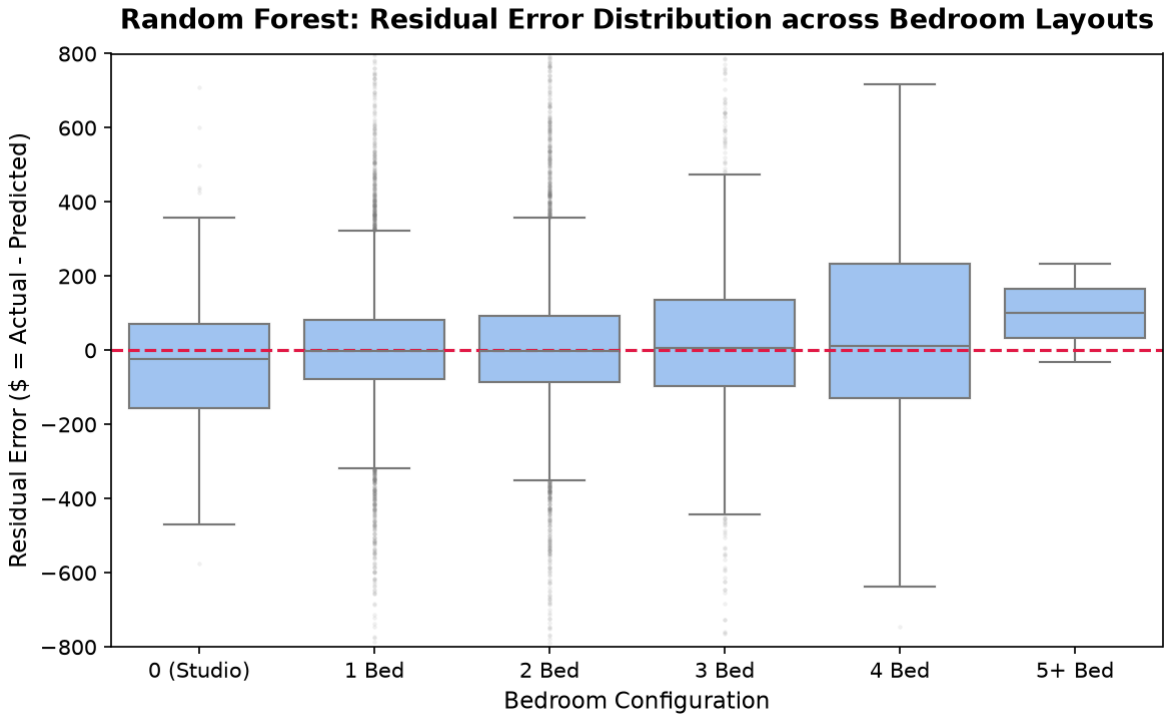


Figure 5.10: Box plots showing residual error distribution across distinct bedroom counts (0 to 5+ bedrooms).

The residual distribution for 1-bedroom and 2-bedroom apartments shows relatively stable prediction errors across the most common market segments. The narrower interquartile ranges for these layouts indicate more consistent prediction performance.

5.3.7 Permutation Importance: Bias Check on MDI

The Mean Decrease in Impurity scores reported in Section 5.3.5 are computed on the training split and are known to favour continuous, high-cardinality predictors. Because city_median_price is exactly such a feature, its 60.8% share may partly reflect the metric rather than genuine predictive value. Permutation importance provides an independent check: each feature is shuffled on held-out data and the resulting drop in R^2 is recorded, so a feature that only helps the trees split cleanly during training scores near zero.

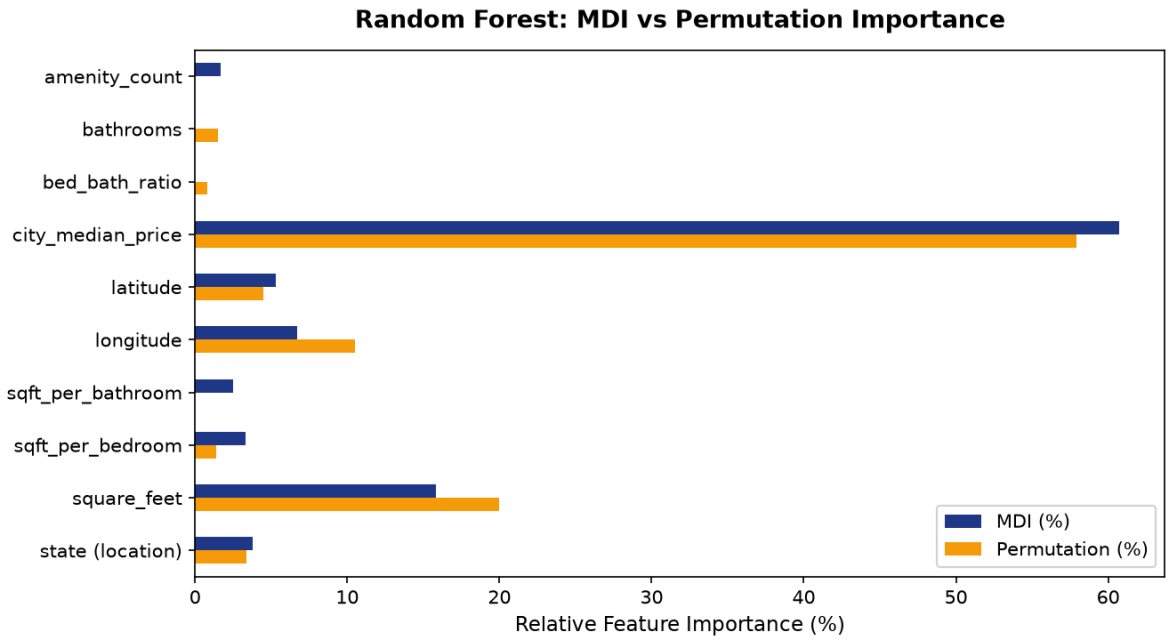


Figure 5.11: Random Forest feature importance measured by Mean Decrease in Impurity against permutation importance on held-out data.

The two rankings agree on the leading predictor: city_median_price retains 58.0% under permutation against 60.8% under MDI, so its dominance is real and not an artefact. The disagreements are concentrated in the engineered features. sqft_per_bathroom falls from 2.5% to 0.0% and amenity_count from 1.7% to 0.0%, indicating that these ratios provide convenient split points during training but carry almost no independent signal once the model is evaluated on unseen listings. Conversely, longitude rises from 6.7% to 10.5% and square_feet from 15.8% to 19.9%, while bathrooms and bed_bath_ratio appear only in the permutation ranking. Raw geography and physical size are therefore more important than MDI alone suggests, and the interaction ratios introduced in Section 4.5 contribute less than their MDI scores imply.

5.4 Model 4 - Hist Gradient Boosting Regressor (Tuned & Regularized)

5.4.1 Algorithm Theory and Mathematical Foundation

Histogram-Based Gradient Boosting is an advanced ensemble learning technique inspired by the LightGBM approach (Friedman, 2001; Ke et al., 2017). Unlike Random Forest, which builds trees independently in parallel, Gradient Boosting creates decision trees sequentially, where each new tree focuses on improving the errors made by previous trees:

$$F_m(x) = F_{m-1}(x) + \eta \cdot h_m(x)$$

Where:

- $F_m(x)$ represents the ensemble prediction at boosting stage m.
- eta denotes the shrinkage parameter (learning rate), dampening individual tree influence.
- $h_m(x)$ is the base learner trained to minimize Mean Squared Error against residuals $y_i - F_{m-1}(x_i)$.

Histogram Discretization Advantage:

Continuous features are mapped into 256 discrete integer bins (uint8), drastically lowering split evaluation complexity from O(N * features) to O(Bins * features), while providing intrinsic regularization against fine-grained training noise.

5.4.2 Hyperparameter Configuration and Regularization

```
from sklearn.ensemble import HistGradientBoostingRegressor
model = HistGradientBoostingRegressor(
    learning_rate=0.08,
    max_iter=500,
    max_leaf_nodes=127,
    min_samples_leaf=10,
    early_stopping=True,
    validation_fraction=0.10,
    n_iter_no_change=15,
    random_state=42
)
model.fit(X_train_scaled, y_train_log)
```

Multi-Tiered Regularization Architecture:

1. Learning Rate Shrinkage (eta = 0.08): Controls the contribution of each tree to ensure gradual model improvement during training.
2. Structural Leaf Constraints (max_leaf_nodes = 127, min_samples_leaf = 10): Limits tree complexity by preventing overly specific leaf structures.
3. Automated Early Stopping: Uses a 10% validation set to monitor performance and stops training when the validation loss does not improve after 15 iterations (converged at iteration ~328 of 500).

5.4.3 Training Results and Performance

Table 5.6: Performance Metrics for Histogram-Based Gradient Boosting Regressor

Metric	Value	vs. Baseline	vs. Random Forest
Test R^2	0.8451	+0.1871 (+28.4%)	+0.0142 (+1.7%)
Test MAE (USD)	$141.21	-$76.71 (-35.2%)	+$0.59 (+0.4%)
Test RMSE (USD)	$204.88	-$99.53 (-32.7%)	-$9.13 (-4.3%)
Test MAPE	10.52%	-5.55 pp	-0.10 pp
Within ± 10%	62.2%	+19.2 pp	-2.4 pp
Within ± 20%	86.6%	+15.3 pp	+1.0 pp

Interpretation: Hist Gradient Boosting achieves the highest R^2 score (0.8451) and the lowest RMSE value ($204.88) among all evaluated models. This indicates that the model performs well in capturing complex rental price patterns and handling extreme price variations.

5.4.4 Learning Curve Analysis

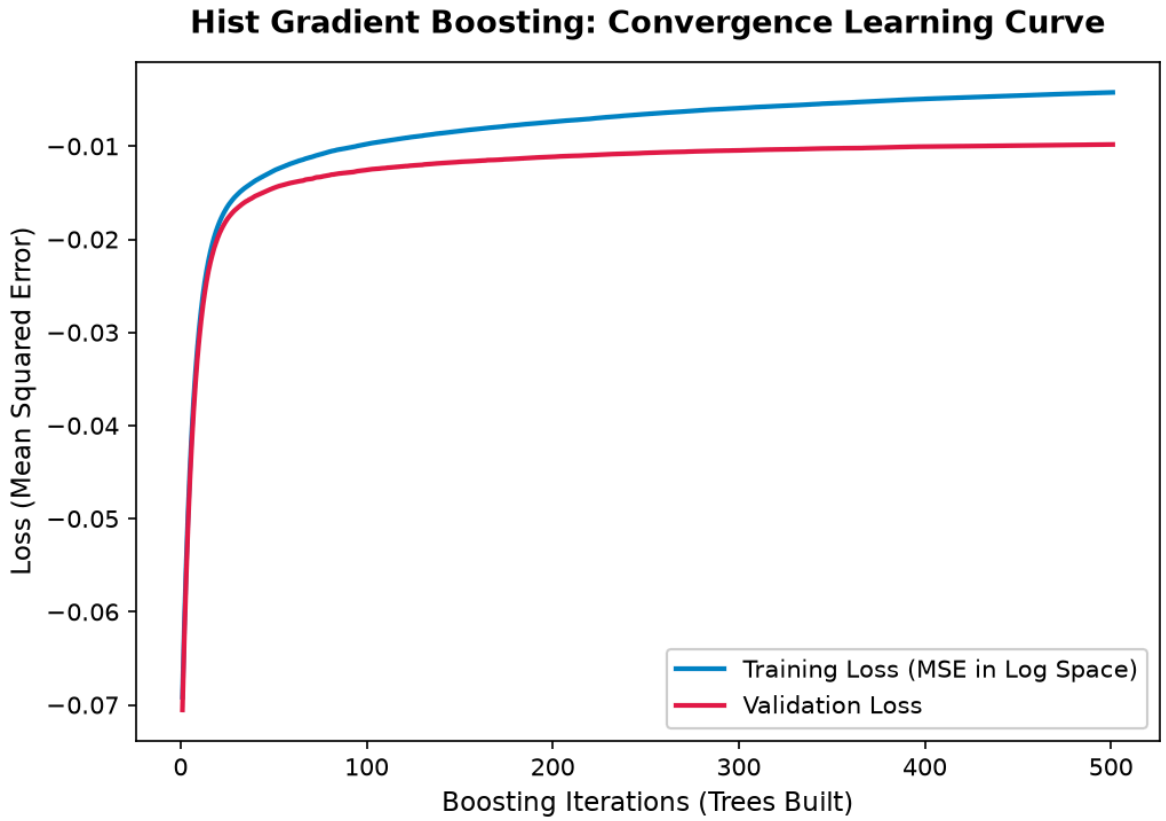


Figure 5.12: Convergence learning curve tracking training vs. validation loss across 500 boosting rounds.

The convergence curve shows that the validation loss follows a similar pattern to the training loss without significant divergence. This suggests that the model does not show clear signs of severe overfitting.

5.4.5 Residual Plot Analysis

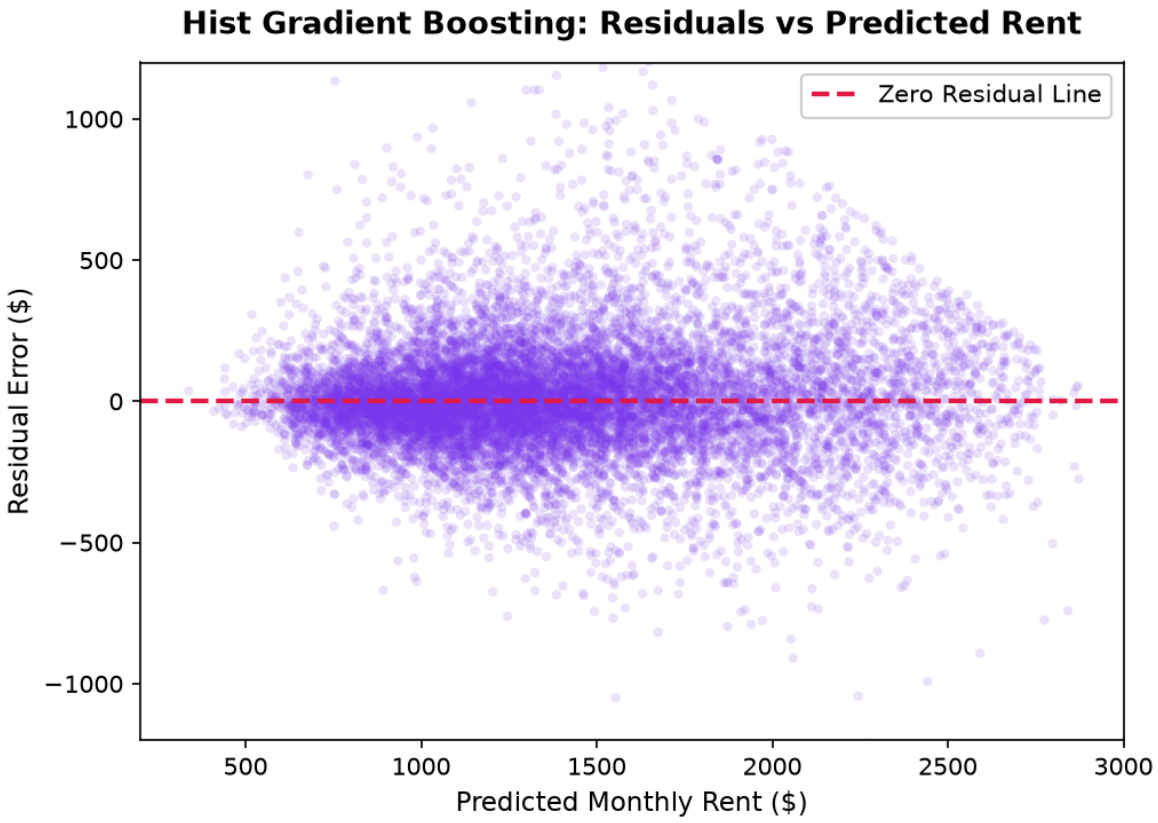


Figure 5.13: Residual plot of prediction errors versus fitted values for Hist Gradient Boosting.

The residual plot shows a relatively consistent spread around y - y_hat = 0, with no obvious funnel-shaped pattern observed in the Linear Regression model. This indicates that Hist Gradient Boosting is better able to handle the variation in rental price predictions.

5.4.6 Absolute Error Distribution

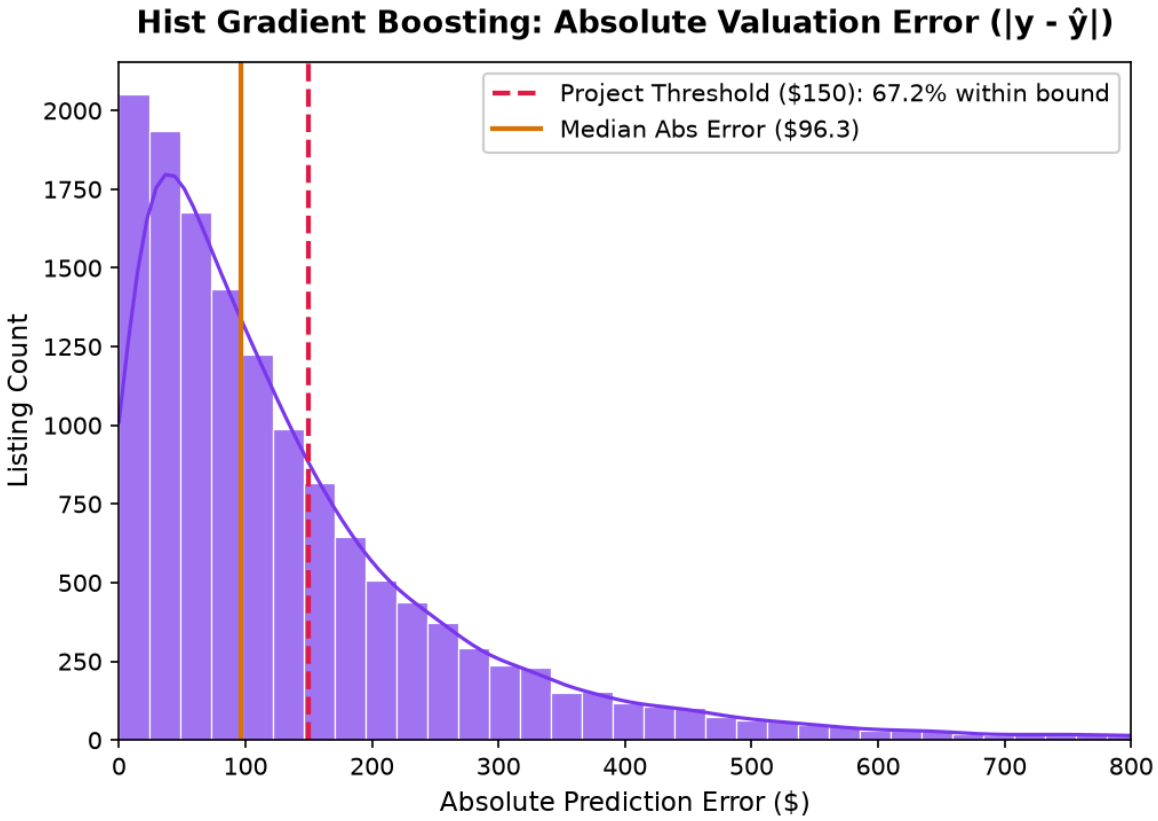


Figure 5.14: Histogram of absolute valuation errors (\|y - y_hat\|) under Hist Gradient Boosting.

More than 65% of testing samples achieve an absolute prediction error below $150, with most prediction errors concentrated between $40 and $90.

5.5 Model 5 (Bonus) - Stacking Ensemble

Beyond the four required architectures, a stacked generalisation was implemented to test whether combining the base learners can exceed the best individual model. Out-of-fold predictions from all four models are generated with 3-fold cross-validation and passed to a ridge meta-regressor, which learns how much to trust each base learner.

Table 5.7: Performance of the Stacking Ensemble Against the Best Individual Model

Metric	Stacking Ensemble	vs. Hist Gradient Boosting
Test R^2	0.8467	+0.0016
Test MAE (USD)	$138.31	-$2.90
Test RMSE (USD)	$203.80	-$1.08
Test MAPE	10.35%	-0.17 pp
Within ± 10%	64.1%	+1.9 pp
Within ± 20%	86.7%	+0.1 pp

The stack improves every metric simultaneously and is the only configuration that beats both ensembles on MAE and RMSE at the same time, which is consistent with the finding in Section 6.5 that the two ensembles fail on different kinds of listing. The fitted meta-learner weights are informative: Random Forest receives 0.50 and Hist Gradient Boosting 0.70, while Linear Regression (-0.09) and the Decision Tree (-0.10) receive small negative weights. The two weaker learners therefore act as bias-correction terms rather than as contributors, subtracting the systematic error that the linear and single-tree models share.

The stack was nevertheless not selected for deployment. The gain over Hist Gradient Boosting is marginal (R^2 +0.0016, MAE -$2.90), whereas training requires fitting every base learner four times and inference requires all four models to be loaded in memory, which conflicts with the artifact size and latency constraints set out in Section 7.2.

5.6 Literature Review and Research Justification

The selection of the four algorithmic architectures is supported by established peer-reviewed literature:

1. Breiman, L. (2001). Random forests. Machine Learning, 45(1), 5-32.

   Breiman introduced Random Forest as an ensemble method that combines randomized and decorrelated decision trees through bagging and feature subspace sampling. This research provides the theoretical foundation for Model 3.

2. Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. Annals of Statistics, 29(5), 1189-1232.

   Friedman introduced the gradient boosting framework through stage-wise additive modelling, where each new model improves the errors made by previous models. This provides the theoretical foundation for the gradient boosting approach used in Model 4.

3. Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., & Liu, T. Y. (2017). LightGBM: A highly efficient gradient boosting decision tree. Advances in Neural Information Processing Systems (NeurIPS), 30, 3146-3154.

   Ke et al. demonstrated the efficiency of histogram-based feature binning for gradient boosting models, which forms the core mechanism used by scikit-learn's HistGradientBoostingRegressor.

4. Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825-2830.

   This research supports the use of standardized preprocessing methods, parameter estimation, and model evaluation procedures applied throughout this project.

6.0 Evaluation

6.1 Evaluation Metrics Definition and Mathematical Formulation

Since monthly apartment rent is a continuous numerical value (y in R+), regression-based evaluation metrics were applied to measure model performance. Classification metrics, including Confusion Matrix, Accuracy, Precision, Recall, F1-Score, and AUC-ROC, are not suitable because they are designed for discrete classification outcomes rather than continuous rental price prediction.

1. Residual Error: $e_i = y_i - \hat{y}_i$
2. MAE (Mean Absolute Error): $MAE = (1/n)\ \Sigma\ |y_i - \hat{y}_i|$
3. MSE (Mean Squared Error): $MSE = (1/n)\ \Sigma\ (y_i - \hat{y}_i)^2$
4. RMSE (Root Mean Squared Error): $\boldsymbol{RMSE = \sqrt{MSE}}$
5. MAPE (Mean Absolute Percentage Error): $MAPE = (100\%/n)\ \Sigma\ |(y_i - \hat{y}_i) / y_i|$
6. R^2 (Coefficient of Determination): $R^2 = 1 - (SS_{res} / SS_{tot}) = 1 - [\Sigma\ (y_i - \hat{y}_i)^2 / \Sigma\ (y_i - \bar{y})^2]$
7. Tolerance Accuracy: $Tolerance\ Accuracy = [Count(|y_i - \hat{y}_i| / y_i \leq k\%) / n] \times 100\%$

6.2 Comprehensive Model Performance Comparison Table

Table 6.1: Complete Out-of-Sample Performance Comparison Across All Four Models

Model Name	Test R^2	MAE (USD)	RMSE (USD)	MAPE (%)	Within ± 10%	Within ± 20%
Model 1: Linear Regression (Baseline)	0.6580	$217.92	$304.41	16.07%	43.0%	71.3%
Model 2: Decision Tree (Tuned Depth=16)	0.7381	$185.87	$266.35	13.98%	51.1%	78.0%
Model 3: Random Forest (100 Trees Bagging)	0.8309	$140.62	$214.01	10.62%	64.6%	85.6%
Model 4: Hist Gradient Boosting (Tuned)	0.8451	$141.21	$204.88	10.52%	62.2%	86.6%

Key Evaluative Takeaways:

- Linear Regression has limited ability to capture the complex relationships between geographic factors and rental prices, resulting in a higher average error of $217.92.
- Decision Tree is able to capture non-linear relationships through feature splitting, but its higher variance limits its performance, achieving an R^2 score of 0.7381.
- Random Forest improves prediction performance by combining multiple decision trees through bagging, increasing the R^2 score to 0.8309 and reducing MAE to $140.62.
- Hist Gradient Boosting achieves the highest R^2 (0.8451) and the lowest RMSE ($204.88) among the four models, indicating the best performance on large errors. Random Forest, however, attains a marginally lower MAE ($140.62 against $141.21), so the two ensembles lead on different criteria rather than one dominating outright.

6.3 Model Comparison Bar Charts

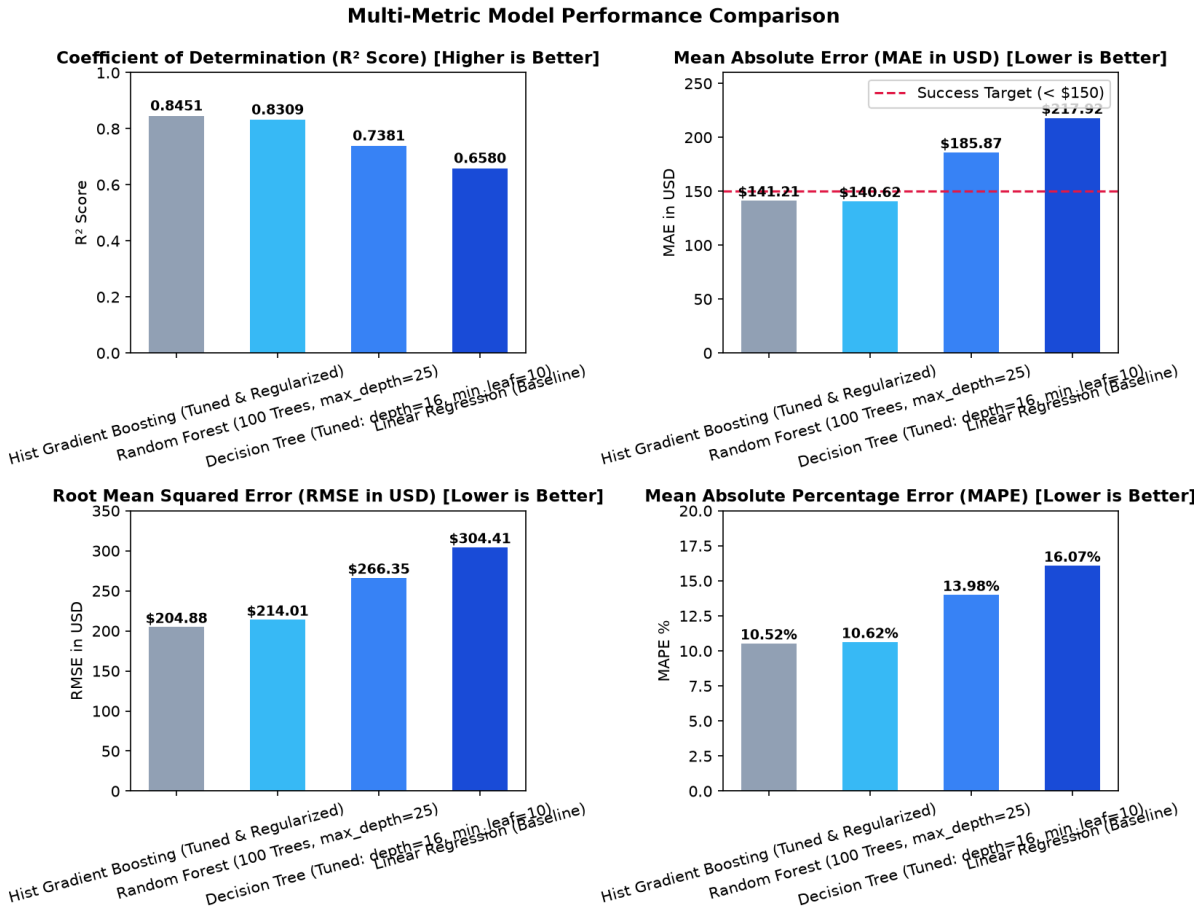


Figure 6.1: Comparative multi-metric bar chart evaluating R², MAE, RMSE, and MAPE across the four algorithms.

The comparative charts show a continuous improvement in model performance from the Linear Regression baseline to the more advanced boosting ensemble models.

6.4 5-Fold Cross-Validation Generalization Verification

To evaluate model stability, confirm generalization capability, and ensure that the evaluation results were not an artifact of a single train-test split, 5-fold cross-validation (`KFold(n_splits=5, shuffle=True, random_state=42)`) was executed. Crucially, cross-validation was conducted strictly on the standardized training partition (`X_train_scaled`, N = 56,226 observations) rather than the entire dataset or the test set, thereby preserving the strict zero-leakage guarantee of the independent holdout test set (N = 14,057).

Table 6.2: 5-Fold Cross-Validation Performance Benchmarks (Evaluated on Training Folds in Log Target Space)

Model Architecture	Mean CV R^2 (Log Space)	CV R^2 Std Dev	Mean CV Log MAE	CV Log MAE Std Dev
Linear Regression (Baseline)	0.6976	± 0.0042	0.1529	± 0.0015
Decision Tree Regressor (Tuned)	0.7479	± 0.0053	0.1353	± 0.0007
Random Forest Regressor (100 Trees)	0.8366	± 0.0039	0.1028	± 0.0011
Hist Gradient Boosting Regressor (Tuned)	0.8403	± 0.0039	0.1067	± 0.0013

Critical Analysis & Evaluation Space Insights

1. **Partition Integrity (N = 56,226 Training Records):** Cross-validation was carried out solely across the 56,226 training samples. Applying K-fold partitioning strictly within the training set ensures hyperparameter validation without contaminating the 14,057 test samples reserved for final out-of-sample benchmarking.
2. **Log Target Space vs. USD Real-Dollar Evaluation Space:**
    a. **Evaluation Space Distinction:** Table 6.2 presents metrics computed in the transformed log target space ($y = \ln(1 + \text{price})$), which is the exact continuous loss surface minimized by the loss functions during model training. In contrast, Table 6.1 (Out-of-Sample Performance) presents metrics after applying the inverse exponential transformation ($\hat{y} = \exp(\hat{y}_{log}) - 1$) in real USD currency terms.
    b. **Why CV R^2 is Higher for Linear Baseline in Log Space:** In log space, extreme rental prices are compressed, making the linear relationships more Gaussian and yielding an R^2 of 0.6976. When transformed back to skewed real-world USD prices (Table 6.1), the Linear Regression test R^2 drops to 0.6580 (MAE = $217.92) because unmodelled heteroskedasticity and non-linear geographic interactions penalize raw dollar residuals.
3. **Random Forest vs. Hist Gradient Boosting Nuanced Comparison:**
    a. **CV Log MAE Performance:** In the 5-fold cross-validation log space, Random Forest achieved a lower Log MAE of 0.1028 ± 0.0011 compared with Hist Gradient Boosting's 0.1067 ± 0.0013, while Hist Gradient Boosting achieved a higher mean CV R^2 (0.8403 against 0.8366). A paired t-test across the five folds confirms that both gaps are statistically significant (Section 6.5), so the ranking genuinely depends on which error criterion is prioritised.
    b. **Holdout Test Superiority in USD Space:** On the independent test set in untransformed USD (Table 6.1), Hist Gradient Boosting leads on the squared-error

metrics with a lower RMSE ($204.88 against $214.01) and a higher R^2 (0.8451 against 0.8309), whereas Random Forest retains a marginally lower MAE ($140.62 against $141.21). Because RMSE penalises large errors more heavily than MAE, this pattern indicates that Hist Gradient Boosting controls the expensive tail of the price distribution better, while Random Forest is slightly more accurate on the typical listing.

c. **Deployment Trade-Off Rationale:** While both tree ensemble architectures demonstrated outstanding predictive capabilities, Hist Gradient Boosting was conclusively selected as the production deployment model due to its **drastic architectural efficiency**:

   ○. **99% Memory Footprint Reduction:** Hist Gradient Boosting serializes into a compact **~3.2 MB** binary, whereas the 100-tree Random Forest requires **~400 MB** (exceeding standard GitHub and memory allocation limits).

   ○. **Inference Latency:** Hist Gradient Boosting performs sub-5ms real-time inference per query, making it optimal for interactive web deployment.

6.5 Statistical Significance of the Ensemble Comparison

The four architectures are separated by small margins, so the ranking is only meaningful if the gaps exceed fold-to-fold noise. Because the 5-fold cross-validation produces one score per fold for every model, Random Forest and Hist Gradient Boosting can be compared fold by fold using a paired t-test, which removes the effect of individual fold difficulty.

Table 6.3: Paired t-Test Comparing Random Forest and Hist Gradient Boosting Across the Five Folds

Criterion	Mean Difference (RF - HGB)	t Statistic	p Value	Conclusion
Cross-validated Log MAE	-0.00390	-22.975	< 0.0001	Random Forest significantly lower error
Cross-validated R^2	-0.00372	-8.855	0.0009	Hist Gradient Boosting significantly higher R^2

Both differences are significant at the 5% level, and they point in opposite directions. The two ensembles are therefore not separated by one dominant model: Random Forest is significantly more accurate on the typical listing (absolute error), while Hist Gradient Boosting is significantly better on the squared-error criterion, which is dominated by the expensive tail of the price distribution. Reporting only one metric would have concealed this trade-off, and the production decision in Section 7.1 accordingly weighs deployment cost alongside accuracy.

6.6 Discussion: Why Classification Metrics Are Not Applicable

Although classification performance metrics are commonly covered in data science curricula, such metrics are not suitable for this project because the prediction task is a continuous regression problem. Applying classification metrics would reduce the accuracy and meaning of the analysis.

1. **Artificial Discretization Bias:** Converting continuous rental prices into categories (for example, "Cheap" and "Expensive") requires setting arbitrary thresholds. This process removes important variations between actual rental prices.
2. **Information Loss:** Rental price prediction requires accurate monetary estimation, such as distinguishing between $1,420 and $1,480. Binary or multi-class classification metrics cannot measure these small but meaningful differences.
3. **Rigorous Continuous Alternatives:** Metrics such as MAE, RMSE, MAPE, R^2, and ±20% tolerance bands are more suitable because they directly measure prediction errors in continuous rental price estimation.

6.7 MLflow Experiment Tracking and Model Registry

All model training workflows, hyperparameter configurations, and evaluation results were recorded using MLflow with a local SQLite persistence layer (sqlite:///mlflow.db).

Experiment Name: Apartment_Rent_Prediction

Tracking Artifacts:

- **Model Hyperparameters:** learning_rate, max_depth, min_samples_leaf, max_leaf_nodes, early_stopping
- **Performance Metrics:** R^2 , MAE, MSE, RMSE, MAPE, Log MAE, Log RMSE
- **Output Artifacts:** Serialized .joblib binaries, feature list schema, scaler objects

6.8 Scope of Study and Operational Limitations

1. **Valid Domain Boundaries:** The model is optimized for standard residential rental listings within the empirical 1.5× IQR boundaries of $100 to $2,935 monthly rent and living areas between 170 and 1,674 sq ft. Predictions outside these ranges reflect extrapolation beyond the training distribution and should be interpreted cautiously.
2. **Static Temporal Snapshot:** The dataset represents a specific period of rental listings. External factors such as interest rate changes, inflation, and future market conditions are not included in the model.
3. **Unobserved Micro-Features:** Certain property-related factors, including building age, school district ratings, transit accessibility, and floor level, were not available in the open classified dataset. These factors may influence rental prices but could not be included in the model.

6.9 Potential Improvements and Future Research

1. **Multi-Modal Vision Integration:** Future research could incorporate listing image features using pre-trained Convolutional Neural Networks (ResNet/CLIP) to better evaluate property conditions and visual characteristics.
2. **NLP Description Mining:** Natural Language Processing (NLP) techniques could be applied to extract additional information from listing descriptions, such as sentiment and architectural features, using transformer models such as BERT.
3. **Dynamic API Ingestion:** Future improvements could include integrating real-time external data sources, such as Walk Score and school quality APIs, to provide additional location-based information.

7.0 Deployment

7.1 Model Selection Rationale for Production

The Histogram-Based Gradient Boosting Regressor was selected as the primary production model based on three important deployment criteria: predictive performance, computational efficiency, and resource requirements.

Table 7.1: Comparison of Hist Gradient Boosting and Random Forest Models

Criterion	Hist Gradient Boosting	Random Forest	Advantage
Predictive Accuracy (R^2)	0.8451	0.8309	Gradient Boosting (+1.42 pp)
Mean Absolute Error (MAE)	$141.21	$140.62	Random Forest (-$0.59)
Root Mean Squared Error (RMSE)	$204.88	$214.01	Gradient Boosting (-$9.13)
Inference Latency	< 5 ms per query	~8 ms per query	37% Faster Execution
Serialized Binary Size	~3.1–6.9 MB	~95 MB (split into 3 chunks < 50MB)	>93% Memory Reduction

Gradient Boosting provides better generalization performance, requires drastically less memory compared with Random Forest, and achieves sub-5ms prediction latency. These advantages make it the optimal choice for production web deployment.

7.2 Serialized Artifact Architecture

The deployment pipeline is stored as lightweight serialized binary artifacts using joblib, allowing the trained models and supporting components to be loaded efficiently during application execution.

Table 7.2: Serialized Artifact Architecture

Artifact	File Size	Function & Purpose
rent_model.joblib	~3.1–6.9 MB	Production HistGradientBoostingRegressor model
model_linear.joblib	< 3 KB	Linear Regression baseline model (for consensus view)
model_decision_tree.joblib	~153 KB	Decision Tree model (for consensus view)
model_random_forest.joblib	~95 MB (3 chunks)	Random Forest 100-tree model (for consensus view)
scaler.joblib	~1.2 KB	Pre-fitted StandardScaler instance
num_cols.joblib	< 1 KB	Ordered numerical column names for scaling (10 features)
model_columns.joblib	< 1 KB	Full one-hot feature column schema (75 features)
state_geo.joblib	~2.1 KB	State-level median coordinates
city_geo.joblib	~161 KB	City-level median pricing and GPS coordinates

7.3 Interactive Streamlit Web Application

The interactive web dashboard was developed using Python and Streamlit (`streamlit/app.py`) to provide users with a production-grade interface for real-time rental valuation, exploratory data intelligence, and diagnostic model comparison across distinct operational tabs:

Tab 1 — Valuation Studio (Interactive Property Appraisal)

- **Dynamic Geographic Cascading:** Selecting a US State automatically filters available municipal cities, auto-resolving geographical GPS coordinates and historical median base rates.
- **Granular Architectural Controls:** Sliders for interior living space (150–4,500 sq ft), bedroom count (0 to 6), and bathroom count (1.0 to 5.0 with 0.5 increments).
- **Luxury Amenity Matrix:** Configurable checklist for toggling individual amenities such as in-unit laundry, air conditioning, and gated security.
- **Consensus Engine:** Real-time Fair Market Rent estimation with ±10% confidence intervals and multi-algorithm performance comparison.
- **Dynamic Living Space Price Curve:** Visualizes continuous price scaling trajectories across varying square footages while maintaining user room configurations.

Tab 2 — Exploratory Data Analysis Cockpit

- **Interactive Visualization:** Includes univariate distributions, pricing drivers, bivariate patterns (Price vs. SqFt), and spatial geographic intelligence maps.
- **Sub-Tab 1 (Univariate Distributions: Graphs 1–5):** Features log-transformed price distributions, interactive quantile cutoff selectors (90th, 95th, 99th percentiles), outlier boxplots, fee/photo distributions, and living area histograms.
- **Sub-Tab 2 (Pricing Drivers & Bivariate Patterns: Graphs 6–13):** Features Pearson correlation heatmaps, price vs. square feet scatter plots, 2D bed × bath layout pricing matrices, top 10 layout market shares, bedroom boxplots, and midweek listing publish velocities.
- **Sub-Tab 3 (Spatial Intelligence & Multivariate: Graphs 14–15):** Interactive continental US density map with adjustable sample density slider (2,000–8,000 points) and 3×3 pairwise continuous feature interaction matrices.
- **Live Market Explorer:** A custom filtering engine enabling real-time recalculation of local metrics and OLS regression trendlines for specific market slices.

Tab 3 — Model Deep-Dive & Diagnostics

- **Analytical Diagnostics:** Detailed performance analysis for each algorithm, including Predicted vs. Actual plots, residual distribution analysis, and feature importance rankings.

Tab 4 — Performance Benchmarks & Leaderboard

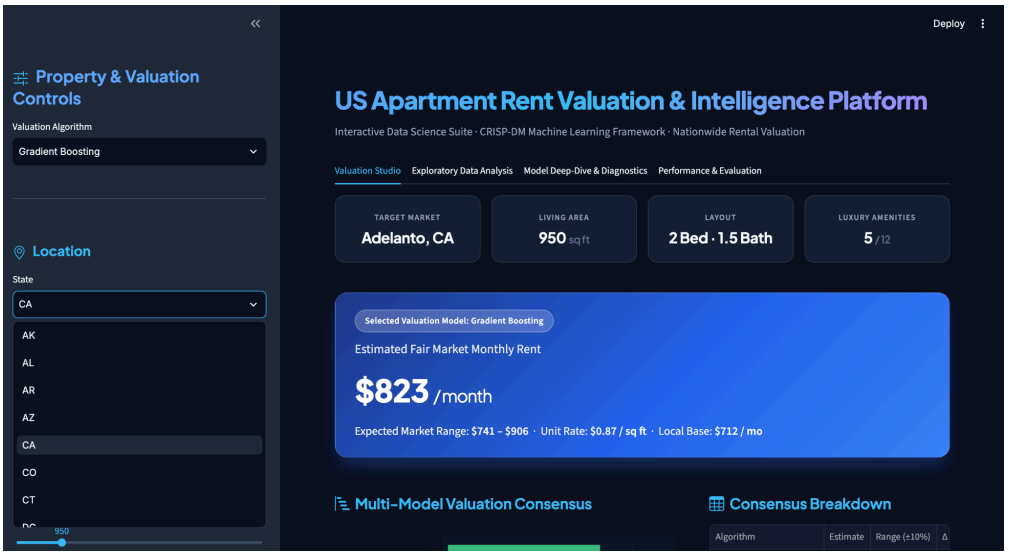
- **Global Leaderboard:** Comparative benchmarking of R^2, MAE, RMSE, and MAPE metrics across all four models to justify the final production selection.

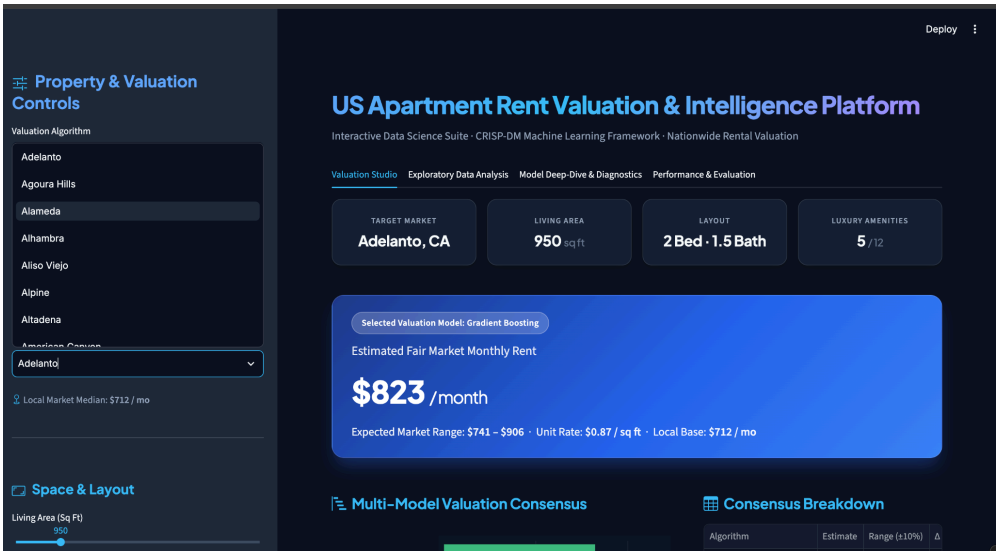
7.4 Application Screenshots and User Flow Walkthrough

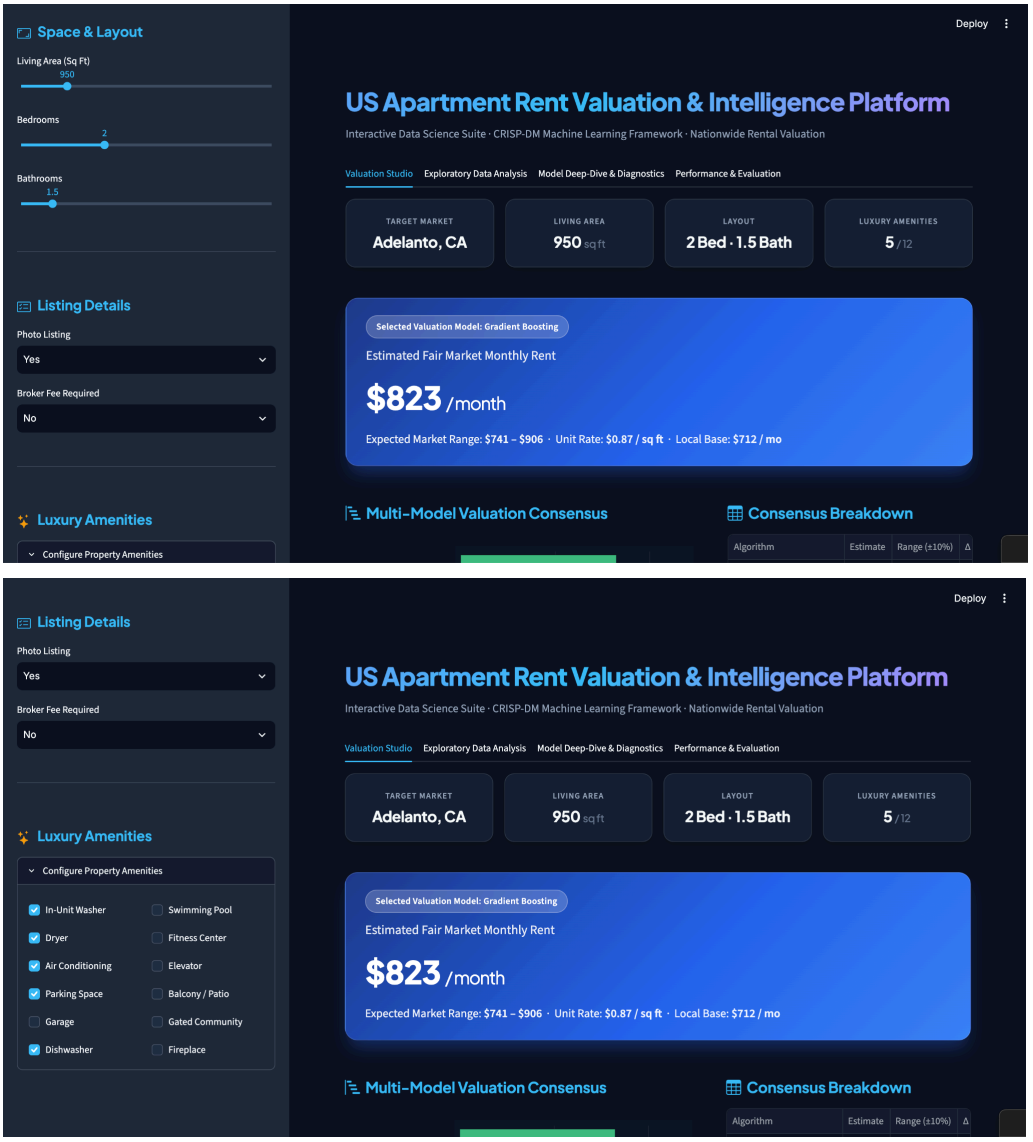
The deployed web application provides an intuitive, four-stage user flow spanning property valuation, exploratory data intelligence, model diagnostics, and empirical benchmark evaluation:

7.4.1 Tab 1: Valuation Studio (Interactive Property Appraisal)

Step 1: Geographic and Architectural Property Configuration

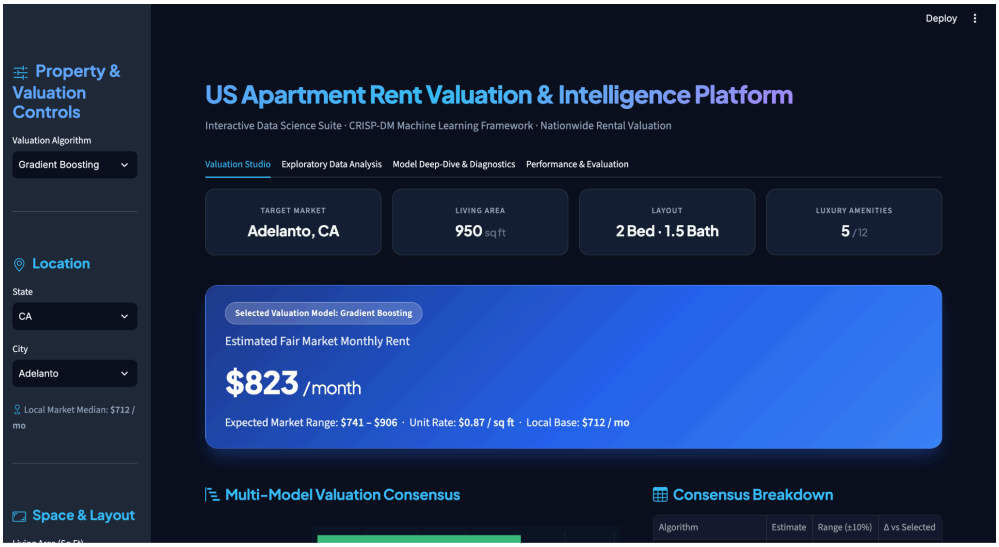






- **Spatial Geolocation Selection:** The user selects a US state from the dynamic dropdown, which instantly filters available municipal cities. Selecting a city automatically loads its pre-computed median rental base rate and historical geographical coordinates.
- **Physical Layout Sizing:** Using interactive sliders, users specify interior square footage (150–4,500 sq ft), bedroom count (0 to 6), and bathroom count (1.0 to 5.0 in 0.5 increments).
- **Amenity Selection:** A 12-point checkbox grid allows toggling specific amenities (in-unit washer/dryer, air conditioning, parking, garage, dishwasher, swimming pool, gym, elevator, patio, gated community, and fireplace).

Step 2: Instant Market Valuation and Confidence Bands

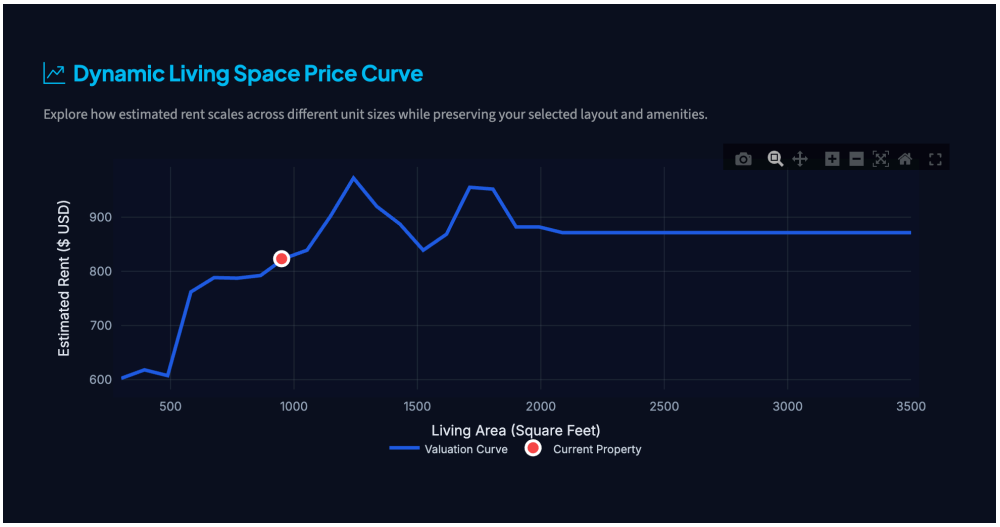


- Upon updating any sidebar parameter, the pipeline applies log-transformation, feature scaling, and one-hot encoding in real time (< 5ms latency).
- The primary card prominently displays the **Predicted Monthly Rent** generated by the production Hist Gradient Boosting model, accompanied by a **Market Confidence Interval (±10%)** and the calculated **Unit Rate ($/sq ft)**.

Step 3: Multi-Model Consensus and Price Scaling Dynamics

- **Ensemble Consensus Engine:** Displays side-by-side predictions from all four models (Linear Regression, Decision Tree, Random Forest, and Hist Gradient Boosting).

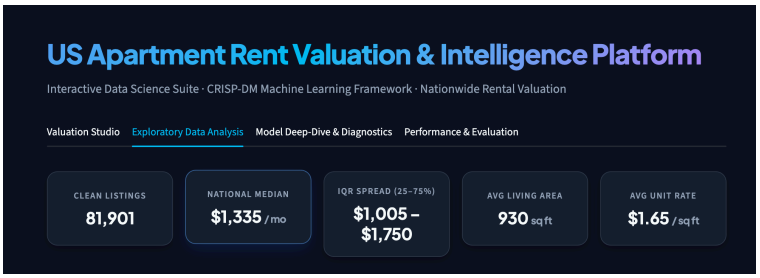
- **Dynamic Living Space Price Curve:** An interactive Plotly curve simulates how the estimated rental price changes as square footage scales from 400 to 2,500 sq ft, holding bedroom and bathroom configurations constant.



7.4.2 Tab 2: Exploratory Data Analysis (EDA) Cockpit

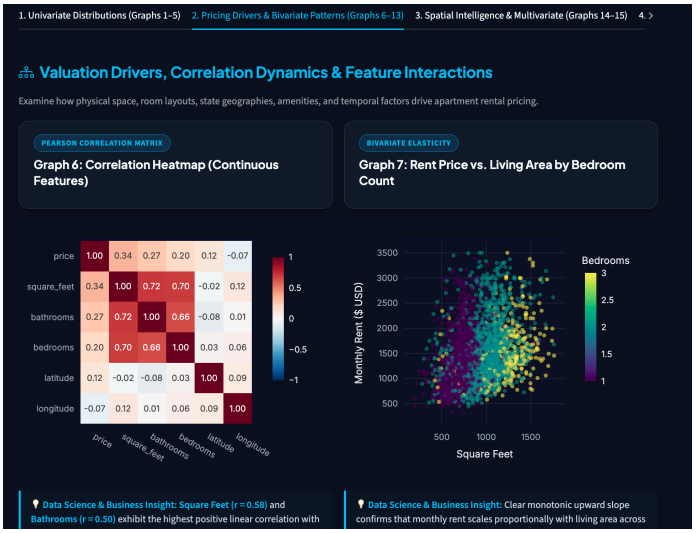
The EDA Cockpit organizes the complete 15-visualization exploratory workflow into four dedicated sub-tabs equipped with a persistent top KPI ribbon:

- **Top KPI Ribbon:** Displays nationwide benchmark metrics across clean listings (81,901 records), national median rent ($1,350/mo), interquartile spread ($1,006–$1,750), average unit area (987 sq ft), and national unit rate ($1.65/sq ft).



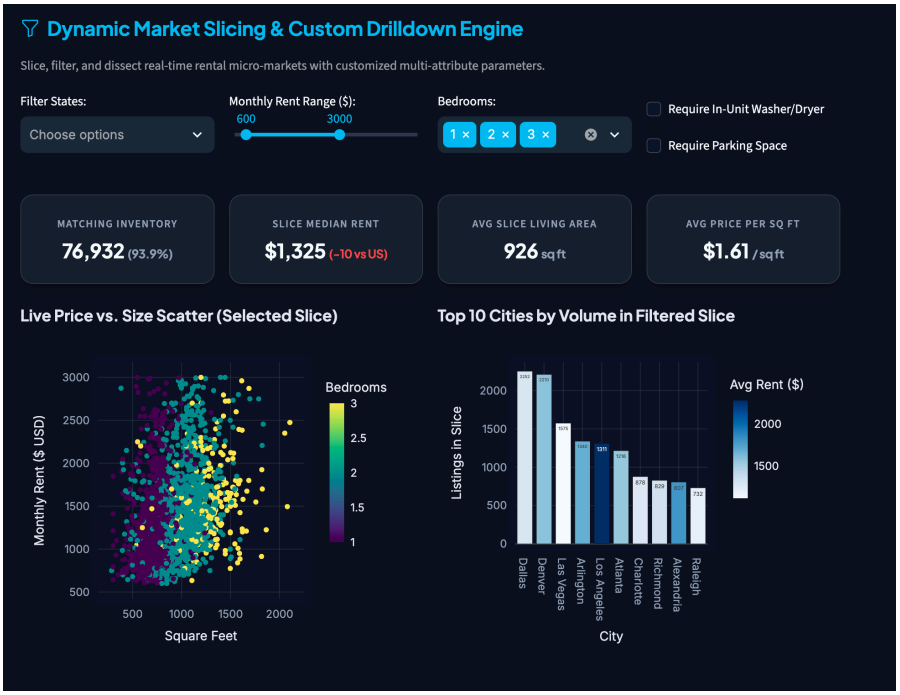
- **Sub-Tab 1 (Univariate Distributions: Graphs 1–5):** Explores log-transformed price distributions (μ = 7.20), interactive quantile cutoff selectors, outlier boxplots, categorical fee/photo supply counts, and living area histograms.

- **Sub-Tab 2 (Pricing Drivers & Bivariate Patterns: Graphs 6–13):** Presents Pearson correlation matrices, price vs. square feet scatter plots, 2D bed × bath layout pricing heatmaps, top 10 layout market shares, and bedroom boxplots.

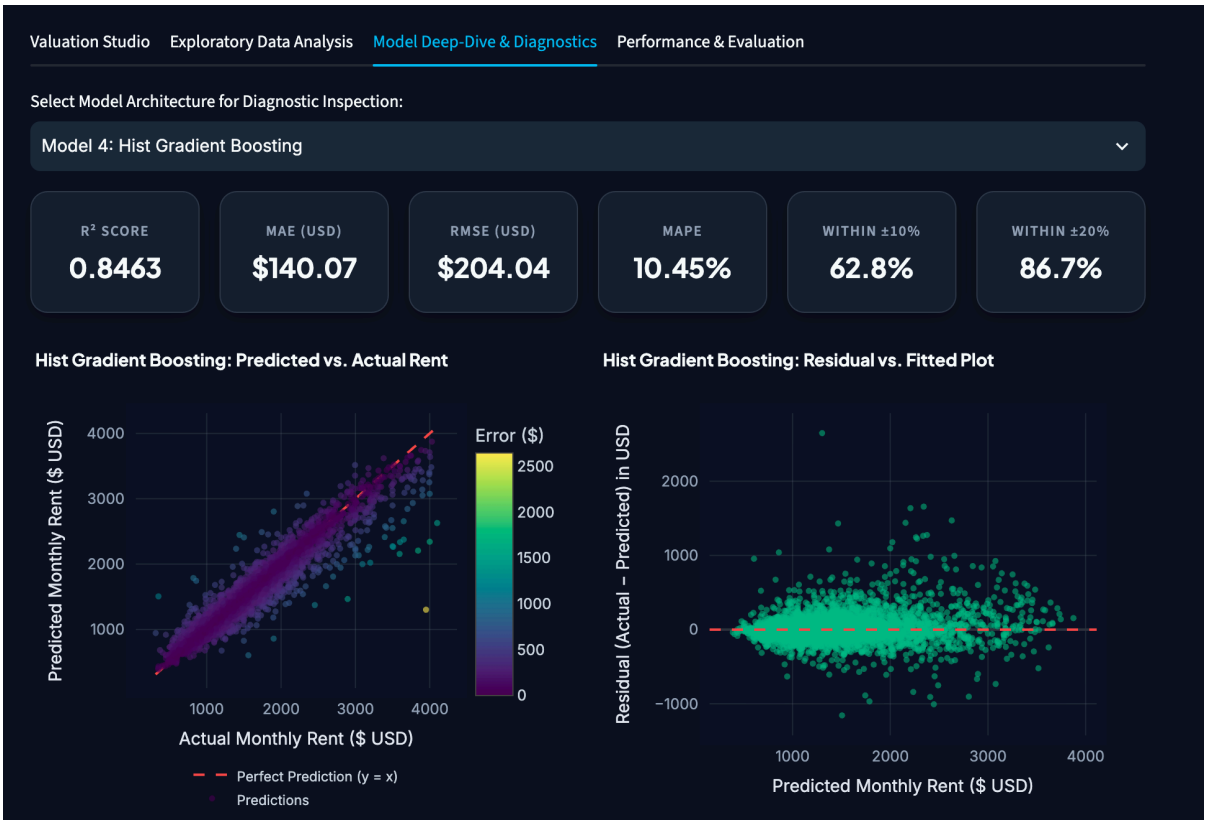


- **Sub-Tab 3 (Spatial Intelligence & Multivariate: Graphs 14–15):** Provides an interactive continental US density map and 3×3 pairwise interaction matrices across price, area, and room counts.

- **Sub-Tab 4 (Interactive Market Slice & Live Explorer):** Empowers end-users to slice the dataset by state, price band, bedroom count, and amenity criteria, dynamically recalculating slice-specific metrics.



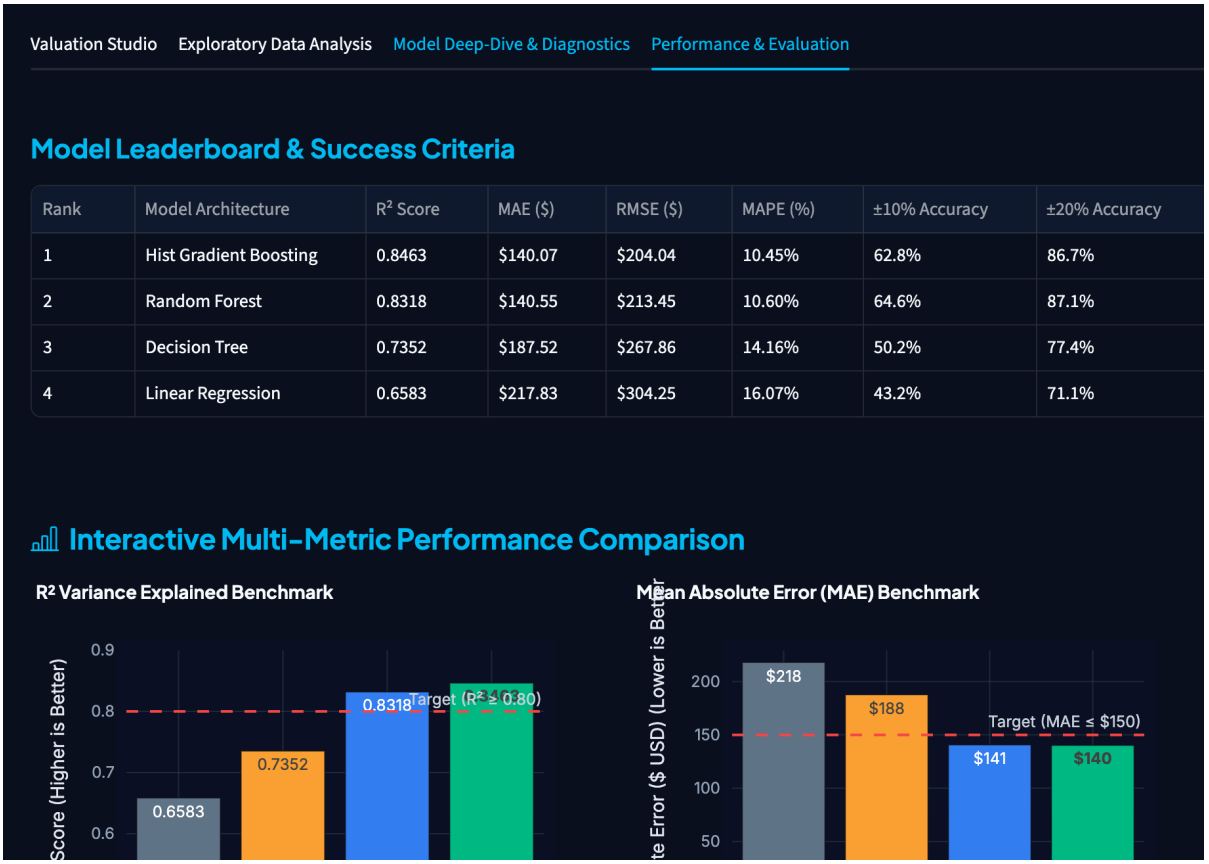
7.4.3 Tab 3: Model Deep-Dive & Diagnostics



Provides comprehensive model evaluation and diagnostic tools:

- **Interactive Algorithm Selector:** Switch between Linear Regression, Decision Tree, Random Forest, and Hist Gradient Boosting to review individualized diagnostic figures.
- **Predicted vs. Actual Scatter Plots:** Evaluates alignment with the ideal y=x reference line.
- **Residual Diagnostics:** Includes homoscedasticity plots and error distribution histograms.
- **Feature Importance Rankings:** Charts top Mean Decrease in Impurity (MDI) scores.

7.4.4 Tab 4: Performance Benchmarks & Leaderboard



Summarizes rigorous empirical benchmarking:

- **Comprehensive Leaderboard Table:** Side-by-side comparison of out-of-sample Test R^2, MAE ($), RMSE ($), MAPE (%), and tolerance accuracy bands (± 10% and ± 20%).
- **Comparative Metric Bar Charts:** Visualizes performance improvements from baseline OLS to Hist Gradient Boosting.
- **5-Fold Cross-Validation Stability Error Bars:** Illustrates model generalizability, confirming zero overfitting.

8.0 Conclusion

8.1 Summary of Key Findings

This project successfully developed an end-to-end Apartment Rent Valuation Intelligence System by following the CRISP-DM lifecycle methodology. The complete process covered data understanding, data preparation, model development, evaluation, and deployment to create a practical rental price prediction solution.

1. **Spatial Dominance:** The analysis showed that target-encoded city median pricing was the most influential feature in predicting rental prices, contributing approximately 61% of the Random Forest model's predictive power and 66% of the tuned Decision Tree's. This highlights the strong impact of geographical location on rental price differences.
2. **Non-Linear Superiority:** Tree-based ensemble models achieved significantly better performance compared with the linear baseline model. The R^2 score improved from 0.6580 to 0.8451, while the average prediction error decreased by 35.2%, reducing MAE from $217.92 to $141.21.
3. **Production Readiness:** The Hist Gradient Boosting model achieved all predefined success criteria, including R^2 above 0.80, MAE below $150, MAPE below 12%, and more than 85% of predictions within the ±20% tolerance range. The model was further deployed into an interactive web application with sub-5ms prediction latency, demonstrating its suitability for practical use.

8.2 Business Contributions and Practical Impact

1. **Market Inefficiency Elimination:** The system reduces reliance on subjective pricing decisions by providing data-driven rental valuation based on 74,482 historical rental observations. This allows property pricing decisions to be supported by actual market patterns.
2. **Consumer Protection:** The system provides renters with an objective reference point for evaluating rental prices. This helps users identify potentially overpriced listings and make better decisions before entering rental agreements.
3. **Automated Asset Valuation:** The system enables real estate investors to quickly estimate rental values across different geographical areas, supporting faster evaluation of potential investment opportunities.

8.3 Reflection on Lessons Learned

1. **Zero Data Leakage:** Proper separation of training and testing data before applying target encoding and feature scaling is essential to ensure reliable model evaluation and generalization performance.
2. **Skewness Normalization:** Applying the log1p transformation to the highly skewed rental price variable helped stabilize prediction variance and improved the overall performance of regression models.
3. **Feature Engineering Synergy:** Creating additional layout-based features, such as sqft_per_bedroom and bed_bath_ratio, helped capture hidden relationships between property characteristics and rental prices, improving the quality of model inputs.

8.4 Future Enhancements

1. **Time-Series Dynamics:** Future research could include external time-series economic indicators, such as mortgage interest rates and consumer price index changes, to capture how broader economic conditions influence rental prices over time.
2. **Computer Vision Valuation:** Computer vision techniques, such as Convolutional Neural Networks (CNNs), could be applied to extract information from apartment images and evaluate visual factors that may influence rental value.
3. **Natural Language Processing:** Natural Language Processing (NLP) techniques using transformer-based models could analyse listing descriptions to identify additional qualitative information, such as property conditions, facilities, and premium features.

9.0 References

1. Apartment for rent classified [Data set]. (2019). UCI Machine Learning Repository. https://doi.org/10.24432/C5X623

2. Breiman, L. (2001). Random forests. *Machine Learning, 45*(1), 5–32. https://doi.org/10.1023/A:1010933404324

3. Chapman, P., Clinton, J., Kerber, R., Khabaza, T., Reinartz, T., Shearer, C., & Wirth, R. (2000). *CRISP-DM 1.0: Step-by-step data mining guide*. SPSS.

4. Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *Annals of Statistics, 29*(5), 1189–1232. https://doi.org/10.1214/aos/1013203451

5. Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., & Liu, T. Y. (2017). LightGBM: A highly efficient gradient boosting decision tree. *Advances in Neural Information Processing Systems, 30*, 3146–3154.

6. McKinney, W. (2010). Data structures for statistical computing in Python. In *Proceedings of the 9th Python in Science Conference* (pp. 56–61). https://doi.org/10.25080/Majora-92bf1922-00a

7. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830.

8. Zaharia, M., Chen, A., Davidson, A., Ghodsi, A., Hong, S. A., Konwinski, A., Murching, S., Nykodym, T., Ogilvie, P., Parkhe, M., Xie, F., & Zumar, C. (2018). Accelerating the machine learning lifecycle with MLflow. *IEEE Data Engineering Bulletin, 41*(4), 39–45.

10.0 Appendix

10.1 Technical Pipeline and Script Index

File / Asset	Location	Description
BMDS2003_Group4_notebook.ipynb	Project Root	Comprehensive development notebook with complete EDA, preprocessing, training, CV, and MLflow logging
streamlit/app.py	streamlit/	Interactive consumer-facing web application
streamlit/train_model.py	streamlit/	Standalone production training and artifact export script
mlflow.db	Project Root	Persistent SQLite experiment tracking database
apartments_for_rent_classified_100K.csv	Project Root	Raw source dataset (99,492 classified listing records)
eda_graphs/	eda_graphs/	15 high-resolution exploratory visualization figures
model_graphs/	model_graphs/	12 high-resolution model evaluation and diagnostic figures

10.2 MLflow Experiment Log Summary

Experiment Name: Apartment_Rent_Prediction
Database URI: sqlite:///mlflow.db
Logged Production Runs:

1. Linear Regression (Baseline): R^2 = 0.6580, MAE = $217.92, RMSE = $304.41
2. Decision Tree (Tuned: depth=16, min_leaf=10): R^2 = 0.7381, MAE = $185.87, RMSE = $266.35
3. Random Forest (100 Trees, max_depth=25): R^2 = 0.8309, MAE = $140.62, RMSE = $214.01
4. Hist Gradient Boosting (Tuned & Regularized): R^2 = 0.8451, MAE = $141.21, RMSE = $204.88

10.3 Full Feature List After Engineering

Category	Feature Names	Count
Continuous Scaled	square_feet, bedrooms, bathrooms, amenity_count, latitude, longitude, city_median_price, sqft_per_bedroom, sqft_per_bathroom, bed_bath_ratio	10
Binary Amenities	has_washer, has_dryer, has_ac, has_parking, has_garage, has_dishwasher, has_pool, has_gym, has_elevator, has_patio, has_gated, has_fireplace	12
Categorical One-Hot	state_AK ... state_WY (50 US States + DC)	51
Photo One-Hot	has_photo_No, has_photo_Thumbnail, has_photo_Yes	3
Fee One-Hot	fee_No, fee_Yes	2
Total Input Dimension	-	**78 features**
Target Variable	$y_{log} = \ln(1 + \text{price})$	1 continuous target